

Chapter 1

Introduction to Kafka

Historically, most applications were built on data-at-rest systems. When some interesting events happened in the world, they were stored in these systems immediately, but the utilization of those events happened later, either when the user explicitly asked for the information, or from some batch-processing jobs that would eventually kick in.

With data-in-motion systems, applications are built by predefining what they want to do when new events occur. When new events happen, they are reflected in the application automatically in near-real time. Such event-driven applications are appealing because they allow enterprises to derive new insights from their data much quicker. Switching to event-driven applications requires a change of mindset, however, which may not always be easy. This book offers a comprehensive resource for understanding event-driven thinking, along with realistic hands-on examples for you to try out

1.1 What is Kafka

The Apache Kafka site (<http://kafka.apache.org/intro>) defines Kafka as a distributed streaming platform. It has three main capabilities:
Reading and writing records like a message queue

Storing records with fault tolerance

Processing streams as they occur [2]

Kafka's message delivery can take at least the following three delivery methods [3]:

- At-least-once semantics—A message is sent as needed until it is acknowledged.
- At-most-once semantics—A message is only sent once and not resent on failure.
- Exactly-once semantics—A message is only seen once by the consumer of the message.

Let's dig into what those messaging options mean. Let's look at at-least-once semantics (figure 1.3). In this case, Kafka can be configured to allow a producer of messages to send the same message more than once and have it written to the brokers. If a message does not receive a guarantee that it was written to the broker, the producer can resend the message [3]. For those cases where you can't miss a message, say that some one has paid an invoice, this guarantee might take some filtering on the consumer end, but it is one of the safest delivery methods

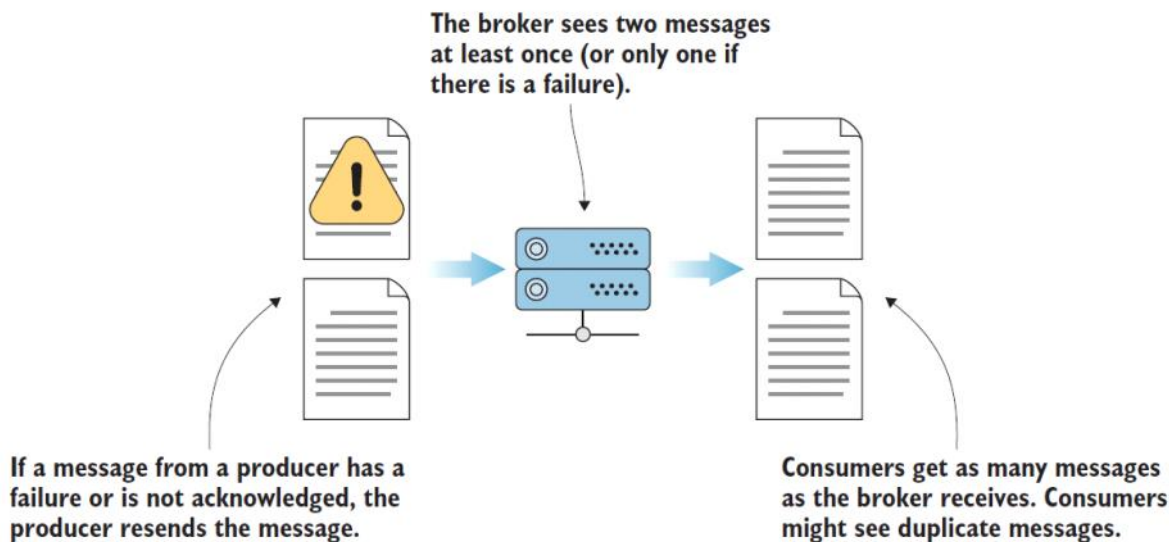


Figure 1.3 At-least-once message flow

At-most-once semantics (figure 1.4) is when a producer of messages might send a message once and never retry. In the event of a failure, the producer moves on and doesn't attempt to send it again [3]. Why would someone be okay with losing a message? If a popular website is tracking page views for visitors, it might be okay with missing a few page view events out of the millions it processes each day. Keeping the system performing and not waiting on acknowledgments might outweigh any cost of lost data

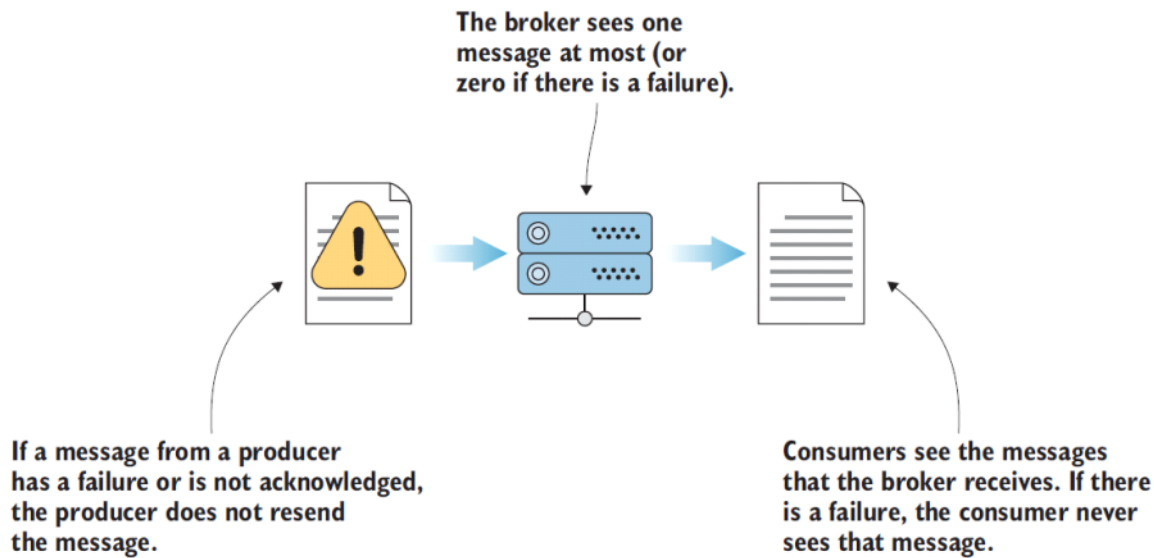


Figure 1.4 At-most-once message flow

Kafka added the exactly-once semantics, also known as EOS, to its feature set in version 0.11.0. EOS generated a lot of mixed discussion with its release [3]. On the one hand, exactly-once semantics (figure 1.5) are ideal for a lot of use cases. This seemed like a logical guarantee for removing duplicate messages, making them a thing of the past. But most developers appreciate sending one message and receiving that same message on the consuming side as well.

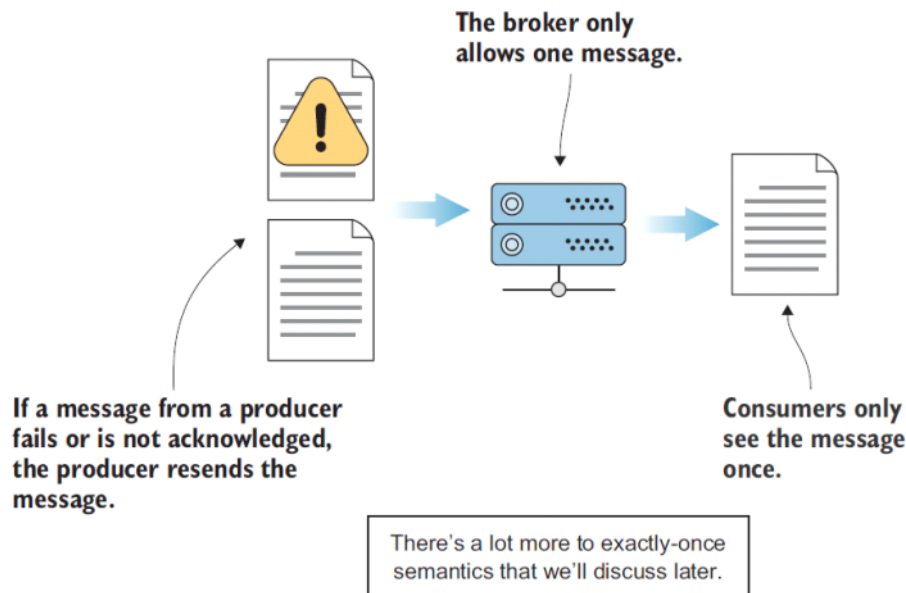


Figure 1.5 Exactly-once message flow

If a producer sends a message more than once, it will still be delivered only once to the end consumer. EOS has touchpoints at all Kafka layers—producers, topics, brokers, and consumers—and will be briefly tackled later in this book as we move along in our discussion for now.

1.2.2 Explaining Kafka to your manager

One of Kafka's most important features is the ability to take volumes of data and make it available for use by various business units. Such a data backbone that makes information coming into the enterprise available to all the multiple business areas allows for flexibility and openness on a company-wide scale. Nothing is prescribed, but increased access to data is a potential outcome.

Most executives also know that with more data than ever flooding in, the company wants insights as fast as possible. Rather than pay for data to get old on disk, its value can be derived as it arrives. Kafka is one way to move away from a daily batch job that limits how quickly that data can be turned into value. Fast data seems to be the newer term, hinting that real value focuses on something different from the promises of big data alone. Kafka can scale horizontally, and not depend on vertical scaling alone, which might eventually reach an expensive peak.

Maybe one of the most important reasons to learn about Kafka is to see how start ups and others in their industry can overcome the once prohibitive cost of computing power. Instead of relying on a bigger and beefier server or a mainframe that can cost millions of dollars, distributed applications and architectures put competitors quickly within reach with, hopefully, less financial outlay

1.3.2 Kafka is the same as other message brokers

Another big myth is that Kafka is just another message broker. Direct comparisons of the features of various tools (such as Pivotal's RabbitMQ™ or IBM's MQSeries®) to Kafka often have asterisks (or fine print) attached and are not always fair to the best use cases of each. Some tools over time have gained or will gain new features just as Kafka has added the exactly-once semantics. And default configurations can be changed to mirror features closer to other tools in the same space. In general, the following lists some of the most exciting and standout features that we will dig into in a bit:

- ▣ The ability to replay messages by default

- ▣ Parallel processing of data

Kafka was designed to have multiple consumers. What that means is that one application reading a message from the message broker doesn't remove it from other applications that might want to consume it as well. One effect of this is that a consumer who has already seen the message can again choose to read it (and other messages as well). With some architecture models like lambda (discussed in chapter 8), programmer mistakes are expected just as much as hardware failures. Imagine consuming millions of messages, and you forget to use a specific field from the original message. In some queues, that message is removed or sent to a duplicate or replay location. However, Kafka provides a way for consumers to seek specific points and read messages again (with a few constraints) by just seeking an earlier position on the topic. As touched on briefly, Kafka allows for parallel processing of data and can have multiple consumers on the same topic. Kafka also has the concept of consumers being part of a consumer group, which will be covered in depth in chapter 5. Membership in a group determines which consumers get which messages and what work has been done across that group of consumers. Consumer groups act independently of any other group and allow for multiple applications to consume messages at their own pace with as many consumers as they require working together. Processing can happen in various ways: consumption by many consumers working on one application or consumption by many applications. No matter what other message brokers support, let's now focus on the robust use cases that have made Kafka one of the options developers turn to for getting work done.

1.4.1 Early examples

Some users' first experience with Kafka (as was mine) was using it as a messaging tool. Personally, after years of using other tools like IBM® WebSphere® MQ (formerly MQ Series), Kafka (which was around version 0.8.3 at the time) seemed simple to use to get messages from point A to point B. Kafka forgoes using popular protocols and standards—like the Extensible Messaging and Presence Protocol (XMPP), Java Message Service (JMS) API (now part of Jakarta EE), or the OASIS® Advanced Message Queuing Protocol (AMQP)—in favor of a custom TCP binary protocol. We will dig in and see some complex uses later.

Oftentimes, memory storage of data in RAM will not be enough to protect your data; if that server dies, the messages are not persisted across a reboot. High availability and persistent storage are built into Kafka from the start. Apache Flume provides a Kafka channel option because the replication and availability allow Flume events to be made immediately available to other sinks if a Flume agent (or the server it is running on) crashes [10]. Kafka enables robust applications to be built and helps handle the expected failures that distributed applications are bound to run into at some point

Log aggregation (figure 1.6) is useful in many situations, including when trying to gather application events that were written in distributed applications. In the figure

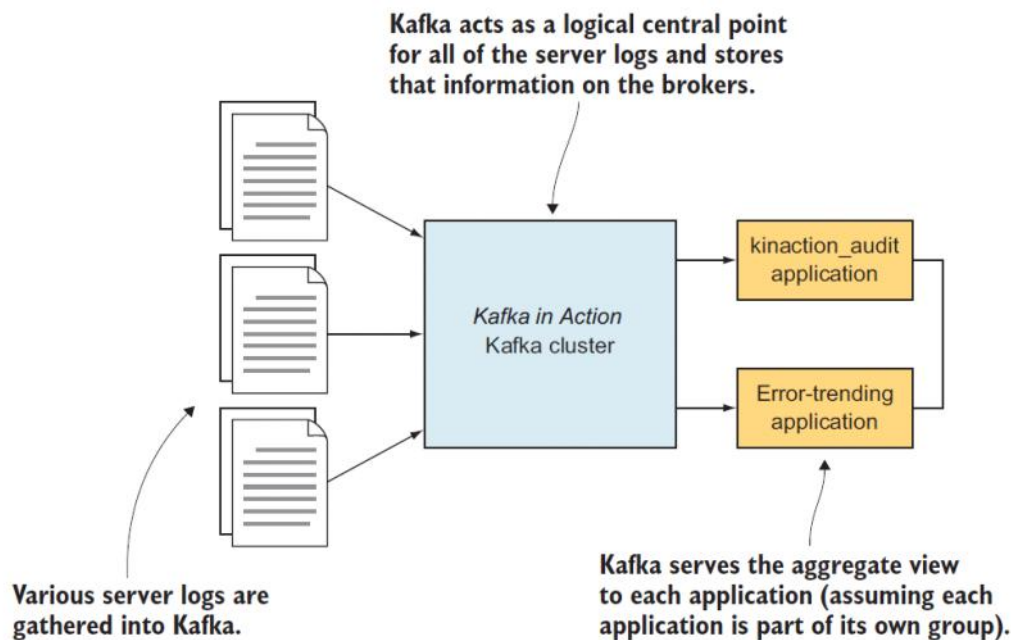


Figure 1.6 Kafka log aggregation

the log files are sent as messages into Kafka, and then different applications have a single logical topic to consume that information. With Kafka's ability to handle large amounts of data, collecting events from various servers or sources is a key feature. Depending on the contents of the log event itself, some organizations use it for auditing and failure-detection trending. Kafka is also used in various logging tools (or as an input option).

1.4.3 When Kafka might not be the right fit

What if you only need a once-monthly or even once-yearly summary of aggregate data? Suppose you don't need an on-demand view, quick answer, or even the ability to reprocess data. In these cases, you might not need Kafka running throughout the entire year for those tasks alone (notably, if that amount of data is manageable to process at once as a batch). As always, your mileage may vary: different users have different thresholds on what is a large batch.

If your main access pattern for data is a mostly random lookup of data, Kafka might not be your best option. Linear read and writes are where Kafka shines and will keep your data moving as quickly as possible. Even if you have heard of Kafka having index files, they are not really what you would compare to a relational database having fields and primary keys from which indexes are built.

Similarly, if you need the exact ordering of messages in Kafka for the entire topic, you will have to look at how practical your workload is in that situation. To avoid any unordered messages, care should be taken to ensure that only one producer request thread is the maximum and, simultaneously, that there is only one partition in the topic. There are various workarounds, but if you have vast amounts of data that depend on strict ordering, there are potential gotchas that might come into play once you notice that your consumption is limited to one consumer per group at a time.

One of the other practical items that come to mind is that large messages are an exciting challenge. The default message size is about 1 MB [13]. With larger messages, you start to see memory pressure increase. In other words, the lower number of messages you can store in page cache could become a concern.

If you are planning on sending huge archives around, you might want to see if there is a better way to manage those messages. Keep in mind that although you can probably achieve your end goal with Kafka in the previous situations (it's always possible), it might not be the first choice to reach for in your toolbox.

Chapter 2

Getting to know Kafka

2.1 Producing and consuming a message

A message, also called a record, is the basic piece of data flowing through Kafka. Messages are how Kafka represents your data. Each message has a timestamp, a value, and an optional key. Custom headers can be used if desired as well [1]. A simple example of a message could be something like the following: the machine with host ID "1234567" (a message key) failed with the message "Alert: Machine Failed" (a message value) at "2020-10-02T10:34:11.654Z" (a message timestamp). Chapter 9 shows an example of using a custom header to set a key-value pair for a tracing use case.

Each key and value can interact in its own specific ways to serialize or deserialize its data.

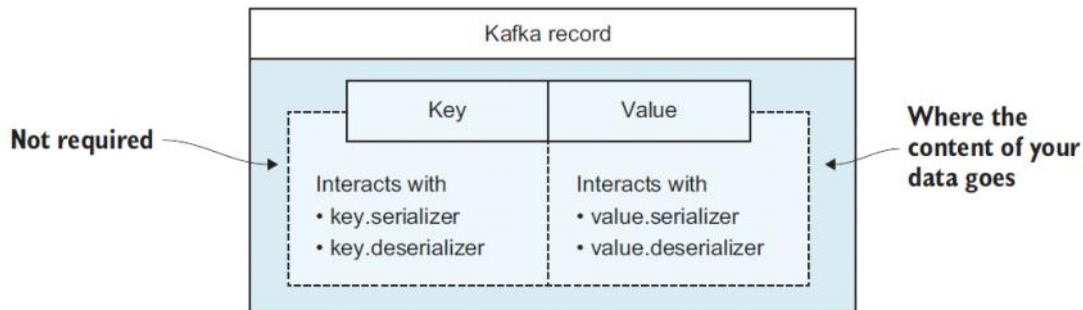


Figure 2.1 Kafka messages are made up of a key and a value (timestamp and optional headers are not shown).

2.2 What are brokers?

Brokers can be thought of as the server side of Kafka [1]. Before virtual machines and Kubernetes®, you may have seen one physical server hosting one broker. Because almost all clusters have more than one server (or node), we will have three Kafka servers running for most of our examples. This local test setup should let us see the output of commands against more than one broker, which will be similar to running with multiple brokers across different machines.

If you are a command line user and want a shortcut to autocomplete commands (and to help with the available arguments), check out a Kafka autocomplete project at <http://mng.bz/K480>. If you are a Zsh user, you may also want to check out and install Kafka's Zsh-completion plugin from <https://github.com/Dabz/kafka-zsh-completions>.

Listing 2.1 Creating the `kinaction_helloworld` topic

```
bin/kafka-topics.sh --create --bootstrap-server localhost:9094
--topic kinaction_helloworld --partitions 3 --replication-factor 3
```

The option determines how many parts we want the topic to be split --partitions into. For example, because we have three brokers, using three partitions gives us one partition per broker. The --replication-factor also is set to three in this example. In essence, this says that for each partition, we want to have three replicas. These copies are a crucial part of our design to improve reliability and fault tolerance. The option points to our local Kafka --bootstrap-server broker. This is why the broker should be running before invoking this script.

Listing 2.3 Describing the topic `kinaction_helloworld`

```
bin/kafka-topics.sh --bootstrap-server localhost:9094 \
--describe --topic kinaction_helloworld
```

--describe lets us look at the details of the topic we pass in.

```
Topic:kinaction_helloworld PartitionCount:3 ReplicationFactor:3 Configs:
Topic: kinaction_helloworld Partition: 0 Leader: 0 Replicas: 0,1,2 Isr: 0,1,2
Topic: kinaction_helloworld Partition: 1 Leader: 1 Replicas: 1,2,0 Isr: 1,2,0
Topic: kinaction_helloworld Partition: 2 Leader: 2 Replicas: 2,0,1 Isr: 2,0,1
```

The output from listing 2.3 shows in the first line a quick data view of the total count of partitions and replicas that this topic has. The following lines show each partition for the topic. The second line of output is specific to the partition labeled and so 0 forth. Let's zoom in on partition 0, which has its replica copy leader on broker 0. This partition also has replicas that exist on brokers 1 and 2. The last column, `Isr`, stands for in-sync replicas. In-sync replicas show which brokers are current and not lagging behind the leader. Having a partition replica copy that is out of date or behind the leader is an issue that we will cover later.

Broker 0 only reads and writes for partition 0. The rest of the replicas get their copies from other brokers.

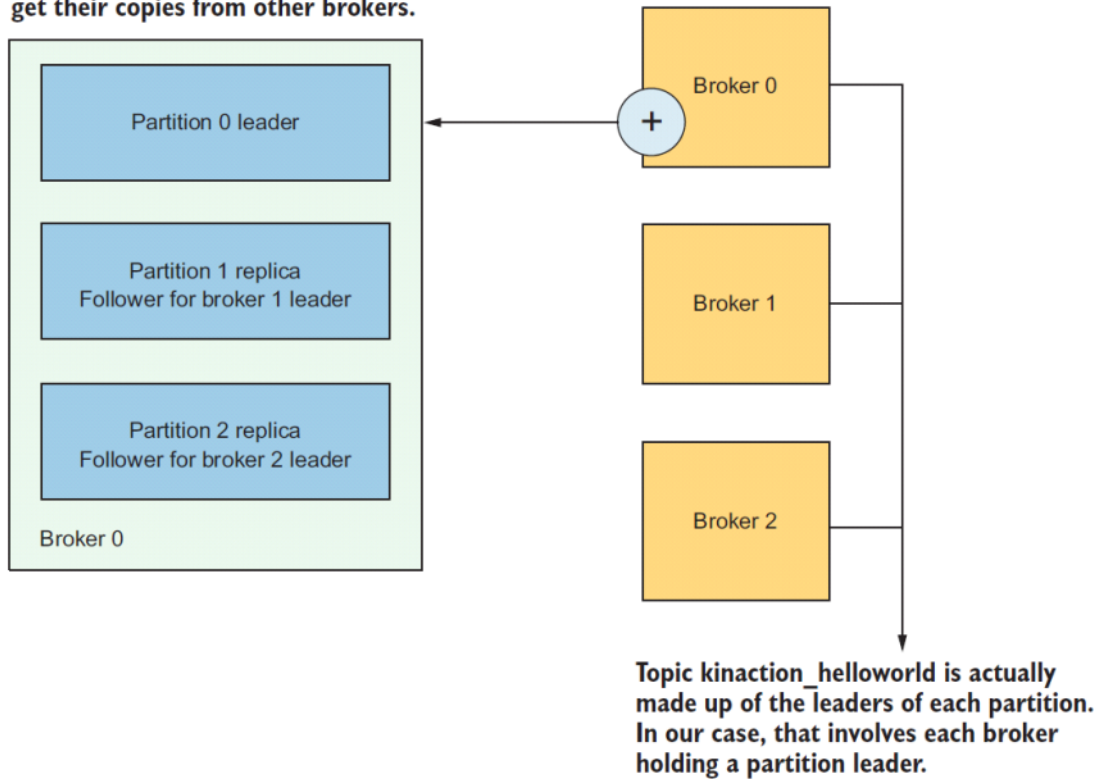
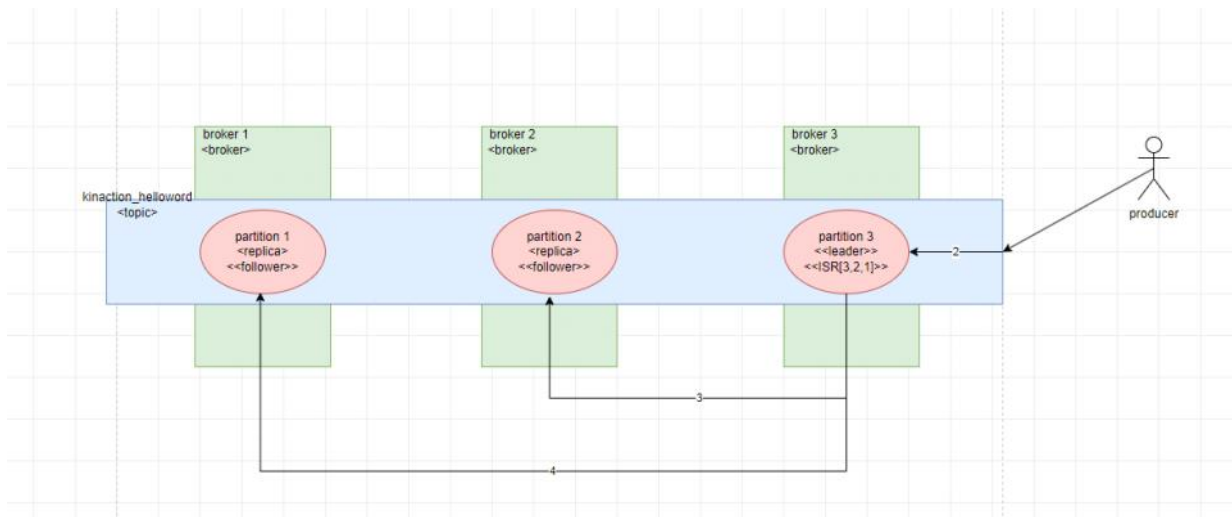


Figure 2.2 View of one broker

Still, it is critical to remember that replica health in a distributed system is something that we will want to keep an eye on. Figure 2.2 shows a view if we look at the one broker with ID 0.



When we reference a partition leader in the image, we are referring to a replica leader. It is important to know that a partition can consist of one or more replicas, but only one replica will be a leader. A leader's role involves being updated by external clients, whereas nonleaders take updates only from their leader.

Listing 2.6 Example consumer output for kinaction_helloworld

```
bin/kafka-console-consumer.sh --bootstrap-server localhost:9094 \  
  --topic kinaction_helloworld --from-beginning  
  
kinaction_helloworld  
...
```

As we send more messages and confirm the delivery to the consumer application, we can terminate the process and eliminate the option when we restart `--from-beginning` it. Notice that we didn't see all of the previously sent messages. Only those messages produced since the consumer console was started show up

WorkerSinkTask is another class inside the Apache Kafka Connect source code from version 1.0 that shows the use of a consumer that is parallel with the producer example from Connect as well (see <http://mng.bz/WrRW>). Consuming applications subscribe to the topics that they are interested in and continuously poll for data. provides a real WorkerSinkTask example in which a consumer is used to retrieve records from topics in Kafka

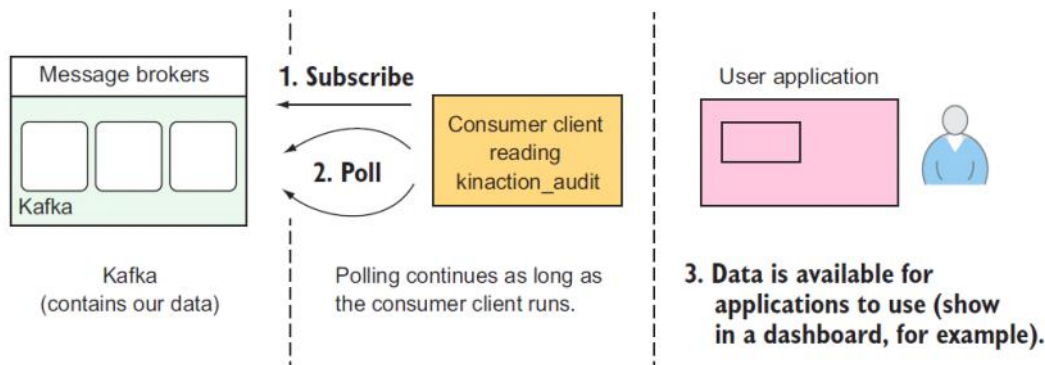


Figure 2.5 Consumer example flow

2.3.2 Topics overview

Topics are where most users start to think about the logic of what messages should go where. Topics consist of units called partitions [1]. In other words, one or more partitions can make up a single topic. As far as what is actually implemented on the computer's disk, partitions are what Kafka works with for the most part. A single partition replica only exists on one broker and cannot be split between brokers.

Figure 2.6 shows how each partition replica leader exists on a single Kafka broker and cannot be divided smaller than that unit. Think back to our first example, the topic. If you're looking `kinaction_helloworld` for reliability and want three copies of the data, the topic itself is not one entity (or a single file) that is copied; instead, it is the various partitions that are replicated three times each.

The topic `kinaction_helloworld` is made up of three partitions that will likely be spread out among different brokers.

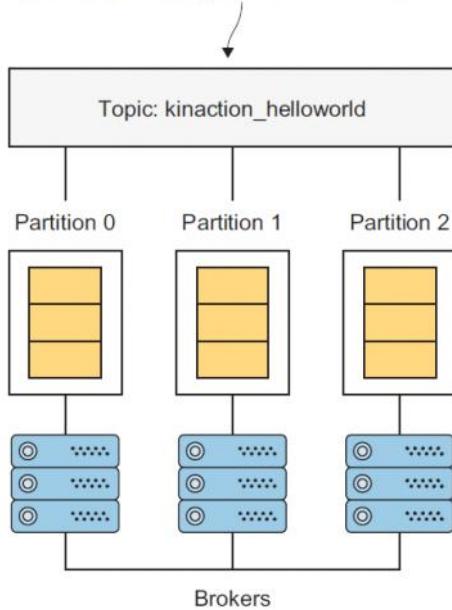


Figure 2.6 Partitions make up topics.

The partition is even further broken up into segment files written on the disk drive. We will cover these files' details and their location when we talk about brokers in later chapters. Although segment files make up partitions, you will likely not interact directly with them, and this should be considered an internal implementation detail.

If you have a topic made up of three partitions and a total of three copies of each partition, every partition will have an elected leader replica. That leader will be one of the copies of the partition, and the other two (in this case, not shown in figure 2.6) will be followers, which update their information from their partition replica leader [1]. Producers and consumers only read or write from the leader replica of each partition it is assigned to during scenarios where there are no exceptions or failures (also known as a "happy path" scenario). But how does your producer or consumer know which partition replica is the leader? In the event of distributed computing and random failures, that answer is often influenced with help from ZooKeeper, the next stop on our tour.

2.3.3 ZooKeeper usage

One of the oldest sources of feared added complexity in the Kafka ecosystem might be that it uses ZooKeeper. Apache ZooKeeper (<http://zookeeper.apache.org/>) is a distributed store that provides discovery, configuration, and synchronization services in a highly available way.

In versions of Kafka since 0.9, changes were made in ZooKeeper that allowed for a consumer to have the option not to store information about how far it had consumed messages (called offsets). We will cover the importance of offsets in later chapters. This reduced usage did not get rid of the need for consensus and coordination in distributed systems, however.

Also, although ZooKeeper will be replaced by the Kafka Raft Metadata mode (KRaft), the concepts of needing coordination in a distributed system are still valid, and understanding the role that ZooKeeper plays currently will, hopefully, lay the foundation of that understanding. Although Kafka provides fault tolerance and resilience, something has to provide coordination, and ZooKeeper enables that piece of the overall system.

As you already saw, our cluster for Kafka includes more than one broker (server). To act as one correct application, these brokers need to not only communicate with each other, they also need to reach an agreement. Agreeing on which one is the replica leader of a partition is one example of the practical application of ZooKeeper within the Kafka ecosystem. For a real-world comparison, most of us have seen examples of clocks getting out of sync and how it becomes impossible to tell the correct time if multiple clocks are showing different times. The agreement can be challenging across separate brokers. Something is needed to keep Kafka coordinated and working in both success and failure scenarios.

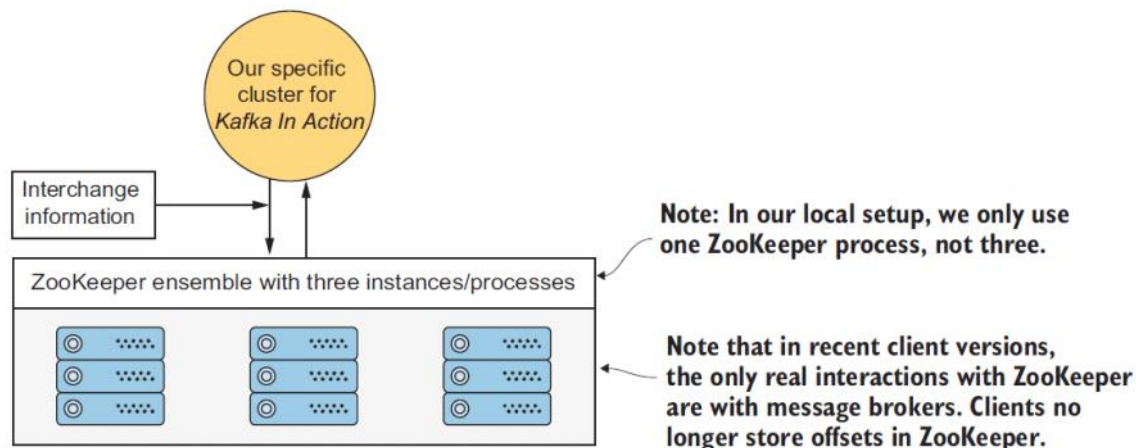


Figure 2.7 ZooKeeper interaction

2.3.4 Kafka's high-level architecture

In general, core Kafka can be thought of as Scala application processes that run on a Java virtual machine (JVM). Although not noted for being able to handle millions of messages quickly, what is it about Kafka's design that makes this possible? One of Kafka's keys is its usage of the operating system's page cache (as shown in figure 2.8). By avoiding caching in the JVM heap, the brokers can help prevent some of the issues that large heaps may have (for example, long or frequent garbage collection pauses) [6]. Another design consideration is the access pattern of data. When new messages flood in, it is likely that the latest messages are of more interest to many consumers, which can then be served from this cache. Serving from a page cache instead of a disk is likely faster in most cases. Where there are exceptions, adding more RAM helps more of your workload to fall into the page cache. As mentioned earlier, Kafka uses its own protocol [7]. Using an existing protocol like AMQP (Advanced Message Queuing Protocol) was noted by Kafka's creators as having too large a part in the impacts on the actual implementation

Tour of Kafka

29

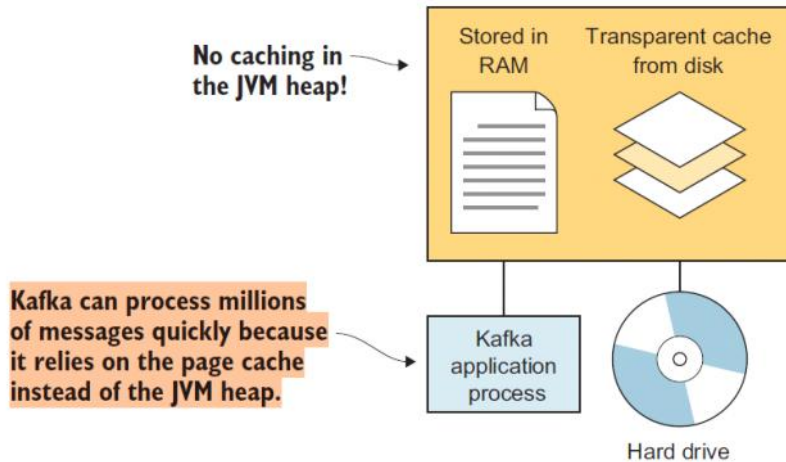


Figure 2.8 The operating system's page cache

2.3.5 The commit log

The log used in Kafka is not just a detail that is hidden in other systems that might use something similar (like a write-ahead log for a database). It is front and center, and its users employ offsets to know where they are in that log. What makes the commit log special is its append-only nature in which events are always added to the end. The persistence as a log itself for storage is a major part of what separates Kafka from other message brokers. Reading a message does not remove it from the system or exclude it from other consumers.

One common question then becomes, how long can I retain data in Kafka? In various companies today, it is not rare to see that after the data in Kafka commit logs hits a configurable size or time retention period, the data is often moved into a permanent store. However, this is a matter of how much disk space you need and your processing workflow. The New York Times has a single partition that holds less than 100 GB [9].

2.4.1 Kafka Streams

Kafka Streams has grabbed a lot of attention compared to core Kafka itself. This API is found in the Kafka source code project's streams directory and is mostly written in Java. One of the sweet spots for Kafka Streams is that no separate processing cluster is needed. It is meant to be a lightweight library to use in your application. You aren't required to have cluster or resource management software like Apache Hadoop to run your workloads. However, it still has powerful features, including local state with fault tolerance, one-at-a-time message processing, and exactly-once support [10]. The more you move throughout this book, the more you will understand the foundations of how the Kafka Streams API uses the existing core of Kafka to do some exciting and powerful work.

This API was made to ensure that creating streaming applications is as easy as possible, and it provides a fluent API, similar to Java 8's Stream API (also referred to as a domain-specific language, or DSL). Kafka Streams takes the core parts of Kafka and works on top of those smaller pieces by adding stateful processing and distributed joins, for example, without much more complexity or overhead.

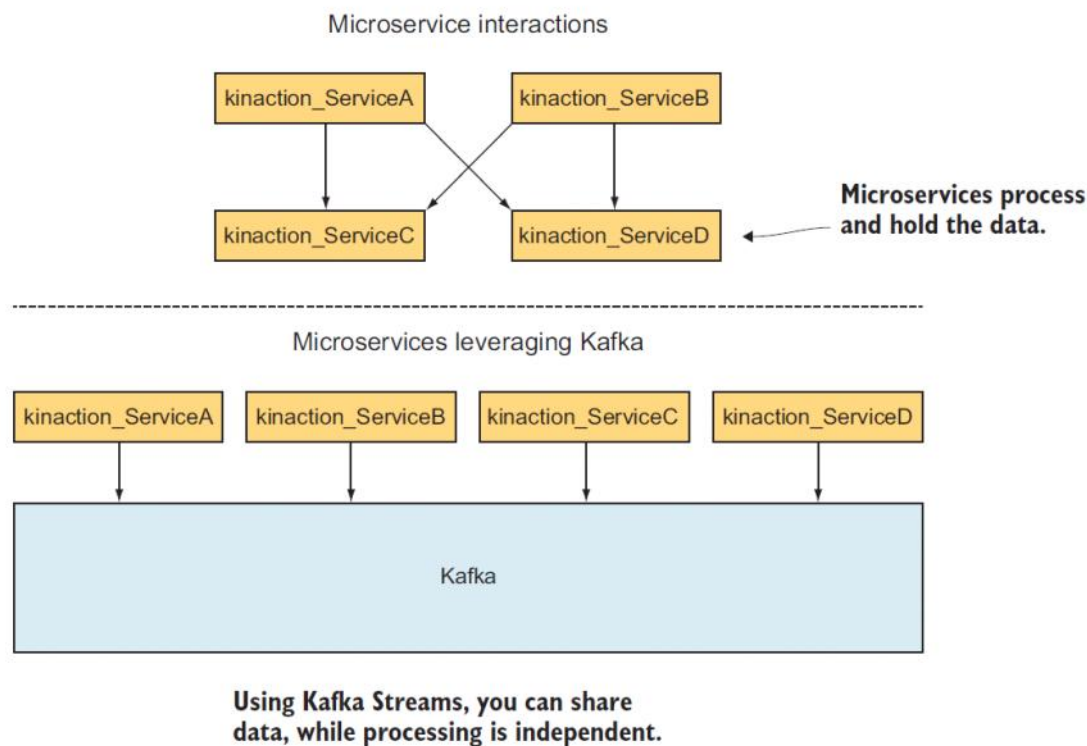


Figure 2.10 Microservice design

2.4.2 Kafka Connect

Kafka Connect is found in the core Kafka Connect folder and is also mostly written in Java. This framework was created to make integrations with other systems easier [10]. In many ways, it can be thought to help replace other tools such as the Apache project Gobblin™ and Apache Flume. If you are familiar with Flume, some of the terms used will likely seem familiar. Source connectors are used to import data from a source into Kafka. For example, if we want to move data from MySQL® tables to Kafka's topics, we would use a Connect source to produce those messages into Kafka. On the other hand, sink connectors are used to export data from Kafka into different systems. For example, if we want messages in some topic to be maintained long term, we would use a sink connector to consume those messages from the topic and place them somewhere like cloud storage

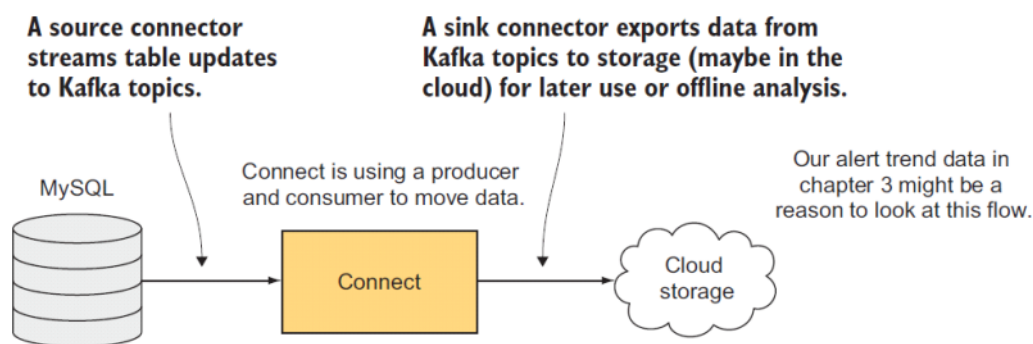


Figure 2.11 Connect use case

2.4.3 AdminClient package

Kafka introduced the AdminClient API recently. Before this API, scripts and other programs that wanted to perform specific administrative actions would either have to run shell scripts (which Kafka provides) or invoke internal classes often used by those shell scripts. This API is part of the kafka-clients.jar file, which is a different JAR than the other APIs discussed previously. The source code can be found in the org/apache/kafka/clients/admin package.

2.4.4 ksqlDB

In late 2017, Confluent released a developer preview of a new SQL engine for Kafka that was called KSQL before being renamed to ksqlDB

Most queries that relational database users are familiar with involve on-demand or one-time queries that include lookups. The mindset shift to a continuous query over a data stream is a significant shift and a new viewpoint for developers. As with the Kafka Streams API, ksqlDB is making it easier to use the power of continuous data flows. Although the interface for data engineers will be a familiar SQL-like grammar, the idea that queries are continuously running and updating is where use cases like dash boards on service outages would likely replace applications that once used point-in time SELECT statements.

2.5 Confluent clients

Listing 2.9 Java client producer

```
public class HelloWorldProducer {
    public static void main(String[] args) {

        Properties kaProperties =
            new Properties();
        kaProperties.put("bootstrap.servers",
            "localhost:9092,localhost:9093,localhost:9094");

        kaProperties.put("key.serializer",
            "org.apache.kafka.common.serialization.StringSerializer");
        kaProperties.put("value.serializer",
            "org.apache.kafka.common.serialization.StringSerializer");

        try (Producer<String, String> producer =
            new KafkaProducer<>(kaProperties))

            ProducerRecord<String, String> producerRecord =
                new ProducerRecord<>("kination_helloworld",
```

The producer takes a map of name-value items to configure its various options.

This property can take a list of Kafka brokers.

Tells the message's key and value what format to serialize

Creates a producer instance. Producers implement the closable interface that's closed automatically by the Java runtime.

The parameter is one essential configuration item, and its purpose bootstrap.servers pose may not be apparent at first glance. This is a list of your Kafka brokers. The list does not have to be every server you have, though, because after the client connects, it will find the information about the rest of the cluster's brokers and not depend on that list.

The key.serializer and value.serializer parameters are also something to take note of in development. We need to provide a class that will serialize the data as it moves into Kafka. Keys and values do not have to use the same serializer.

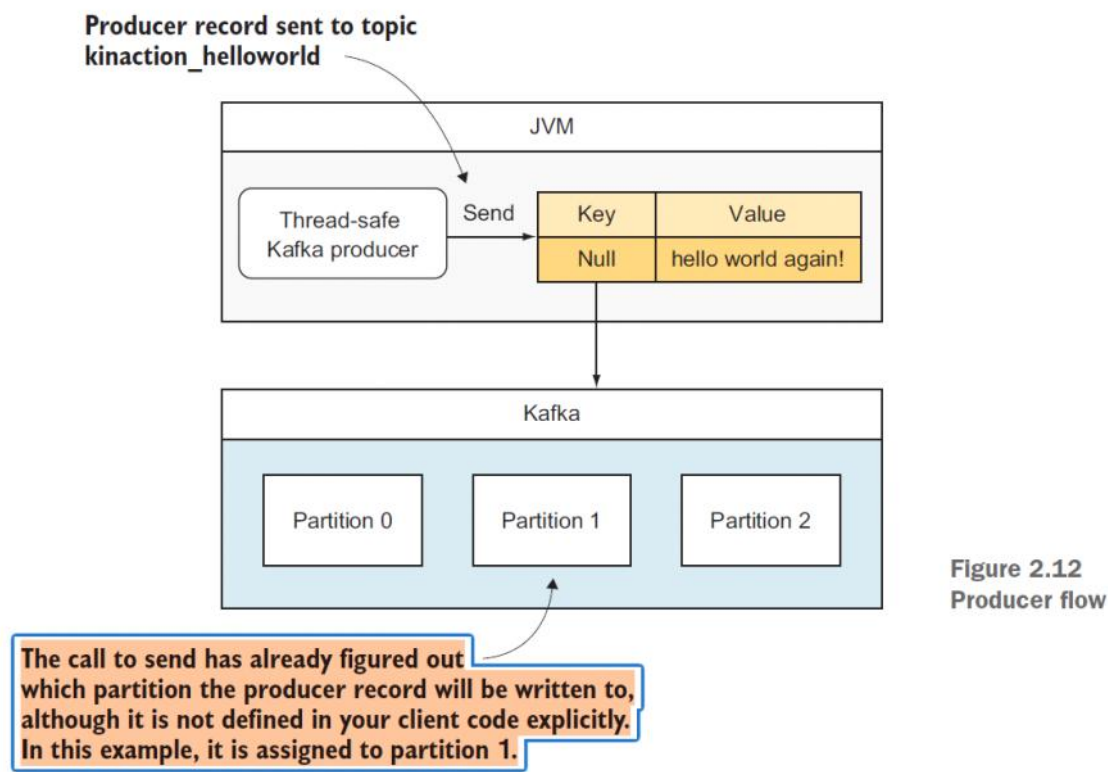


Figure 2.12
Producer flow

Listing 2.11 Java client consumer

```
public class HelloWorldConsumer {

    final static Logger log =
        LoggerFactory.getLogger(HelloWorldConsumer.class);

    private volatile boolean keepConsuming = true;

    public static void main(String[] args) {
        Properties kaProperties = new Properties();
        kaProperties.put("bootstrap.servers",
            "localhost:9092,localhost:9093,localhost:9094");
        kaProperties.put("group.id", "kination_helloconsumer");
        kaProperties.put("enable.auto.commit", "true");
        kaProperties.put("auto.commit.interval.ms", "1000");
        kaProperties.put("key.deserializer",
            "org.apache.kafka.common.serialization.StringDeserializer");
        kaProperties.put("value.deserializer",
            "org.apache.kafka.common.serialization.StringDeserializer");

        HelloWorldConsumer helloWorldConsumer = new HelloWorldConsumer();
        helloWorldConsumer.consume(kaProperties);
        Runtime.getRuntime().
            addShutdownHook(new Thread(helloWorldConsumer::shutdown));
    }

    private void consume(Properties kaProperties) {
        try (KafkaConsumer<String, String> consumer =
            new KafkaConsumer<>(kaProperties)) {
            consumer.subscribe(
                List.of(
                    "kination_helloworld"
                )
            );

            while (keepConsuming) {
                ConsumerRecords<String, String> records =
                    consumer.poll(Duration.ofMillis(250));
                for (ConsumerRecord<String, String> record :
                    records) {
                    log.info("kination_info offset = {}, kination_value = {}",
                        record.offset(), record.value());
                }
            }
        }

        private void shutdown() {
            keepConsuming = false;
        }
    }
}
```

Properties are set the same way as producers.

The consumer tells Kafka what topics it's interested in.

Polls for new messages as they come in

To see the result, prints each record that we consume to the console

However, unlike the producer, the Java consumer client is not thread safe [15]. We will need to take that into account as we scale past one consumer in later sections. Our code is responsible for ensuring that any access is synchronized: one simple option is having only one consumer per Java thread. A command can subscribe to more than one topic at a time. subscribe

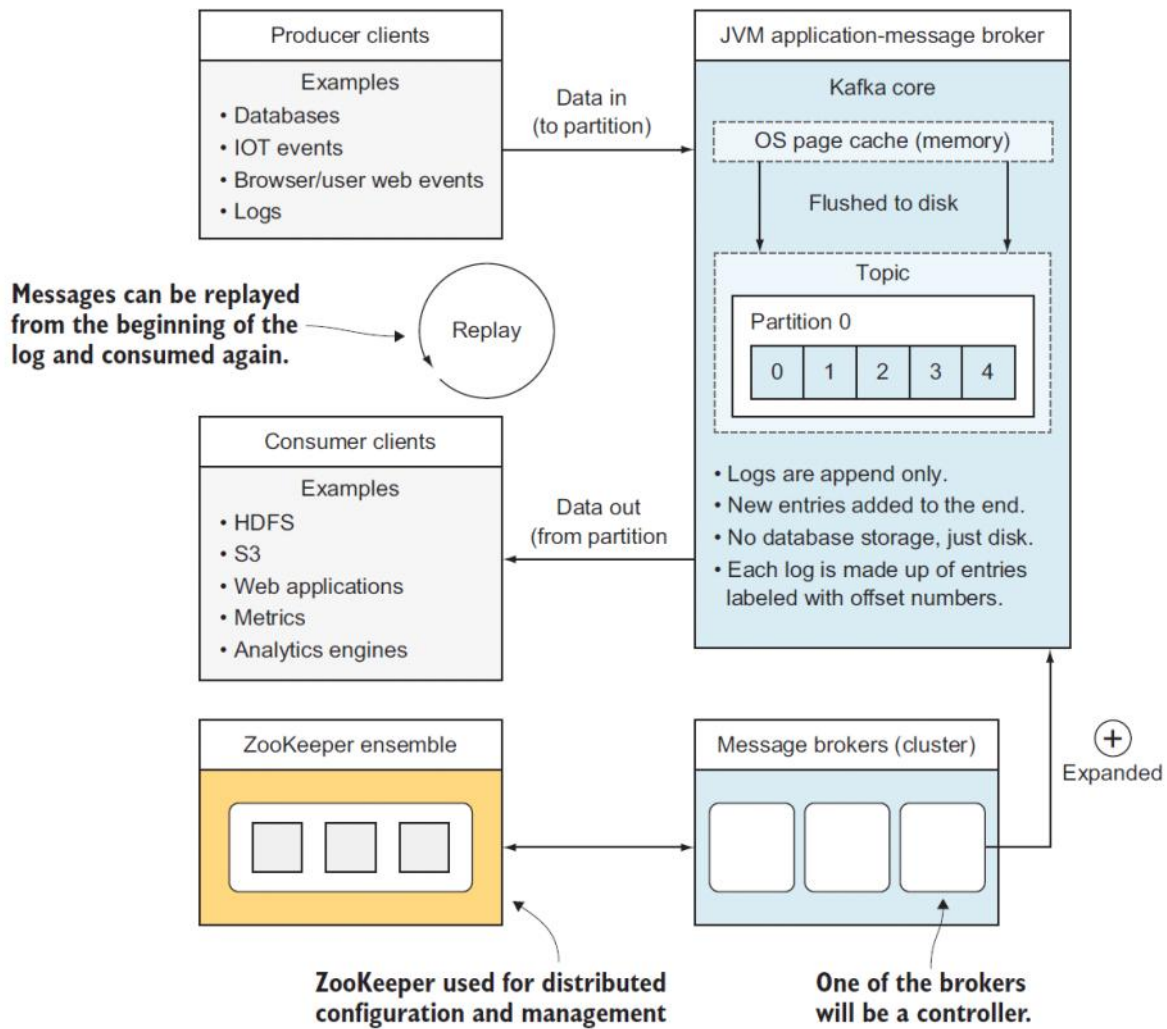


Figure 2.13 Kafka overview

Producers send data into Kafka, which works as a distributed system for reliability and scale, with logs, which are the basis for storage. Once data is inside the Kafka ecosystem, consumers can help users utilize that data in their other applications and use cases. Our brokers make up the cluster and coordinate with a ZooKeeper cluster to maintain metadata. Because Kafka stores data on disk, the ability to replay data in case of an application failure is also part of Kafka's feature set. These attributes allow Kafka to become the foundation of powerful stream-processing applications.

2.6.1 Stream processing

Stream processing seems to have various definitions throughout various projects. The core principle of streaming data is that data will keep arriving and will not end. This approach does not batch data and then process it in groups. The idea of a nightly or monthly run is also not a part of this workflow. If you think of a never-ending waterfall, the same principles apply. Sometimes there is a massive amount of data to transit and sometimes not that much, but it continuously flows between destinations.

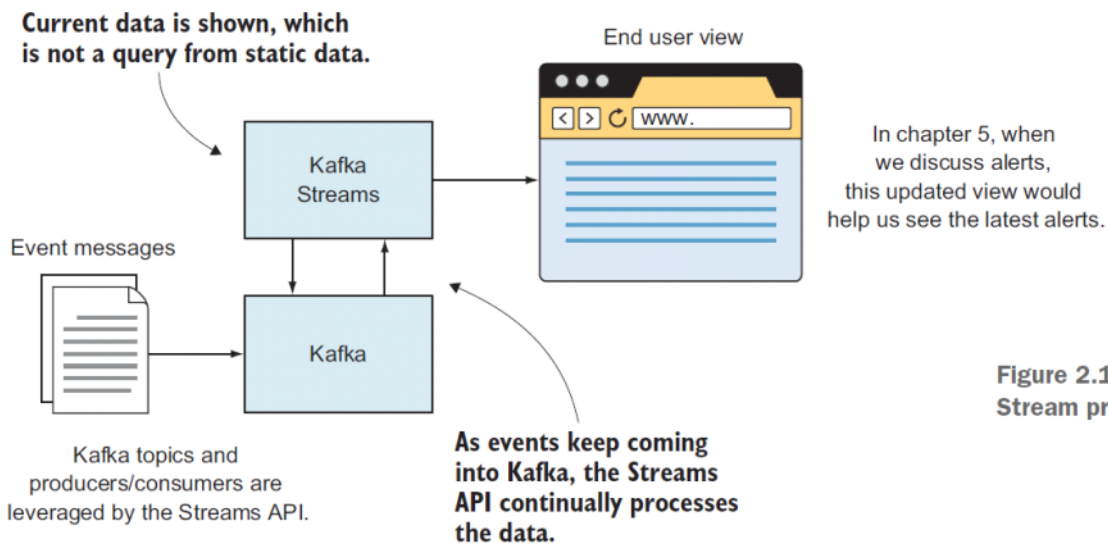


Figure 2.14
Stream process

2.6.2 What exactly-once means

One of the most exciting and maybe most discussed features in Kafka is its exactly once semantics. This Having a closed system that can be completed as a transaction is why using the Streams API is one of the easiest paths to exactly-once

Summary

- Kafka's use of a commit log helps in understanding the system overall

Chapter 3

Designing a Kafka project

Chapter 4

Producers: Sourcing data

Because this distributed system is designed to account for momentary errors such as a network blip, the logic for retries is already built in. However, if the ordering of the messages is essential, like for our audit messages, then besides setting the retries to a number like 3, we also need to set the `max.in.flight.requests.per.connection` value to 1 and set `acks` (the number of brokers that send acknowledgments back) to all [3] [4]. In our opinion, this is one of the safest methods to ensure that your producer's messages arrive in the order you intend [4]. We can set the values for both `acks` and `retries` as configuration parameters.

Another option to be aware of is using an idempotent producer. The term idempotent refers to how sending the same message multiple times only results in producing the message once. To use an idempotent producer, we can set the configuration property `enable.idempotence=true` [5]. We will not be using the idempotent producer in our following examples.

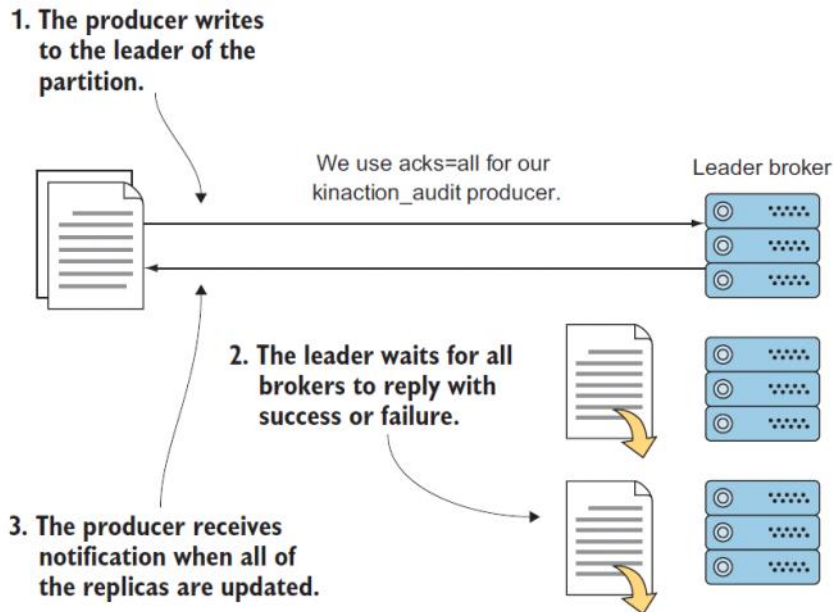


Figure 4.6 The property `acks` equals `all`.

In many cases, it is worth paying the performance price in order to prevent data loss. With many brokers in a cluster, we need to be aware of the number of brokers the leader has to wait on. The broker that takes the longest to reply is the determining factor for how long until a producer receives a success message.

Figure 4.7 shows the impact of setting the `acks` value to 1 and asking for an acknowledgment. An acknowledgment involves the receiver of the message (the leader replica of the specific partition) sending confirmation back to the producer. The producer client waits for that acknowledgment. However, the followers might not have copied the message before a failure brings down the leader. If that situation occurs before a copy is made, the message never appears on the replica followers for that partition [9]. Figure 4.7 shows that while the message was acknowledged by the leader replica and sent to the producer, a failure of any replica to make a copy of the message would appear as if the message never made it to the cluster.

Figure 4.7 The property `acks` equals 1.

NOTE This is closely related to the ideas of at-most and at-least semantics that we covered in chapter 1 [10]. The `acks` setting is a part of that larger picture.

4.2.3 Timestamps

Recent versions of the producer record contain a timestamp on the events you send. A user can either pass the time into the constructor as a Java type `long` when sending a `ProducerRecord` Java object or the current system time. The actual time that is used in

3. Before the leader that has success has time to copy the message to any follower replicas, the leader fails. This means that the message could be lost to the remaining brokers.

4. These

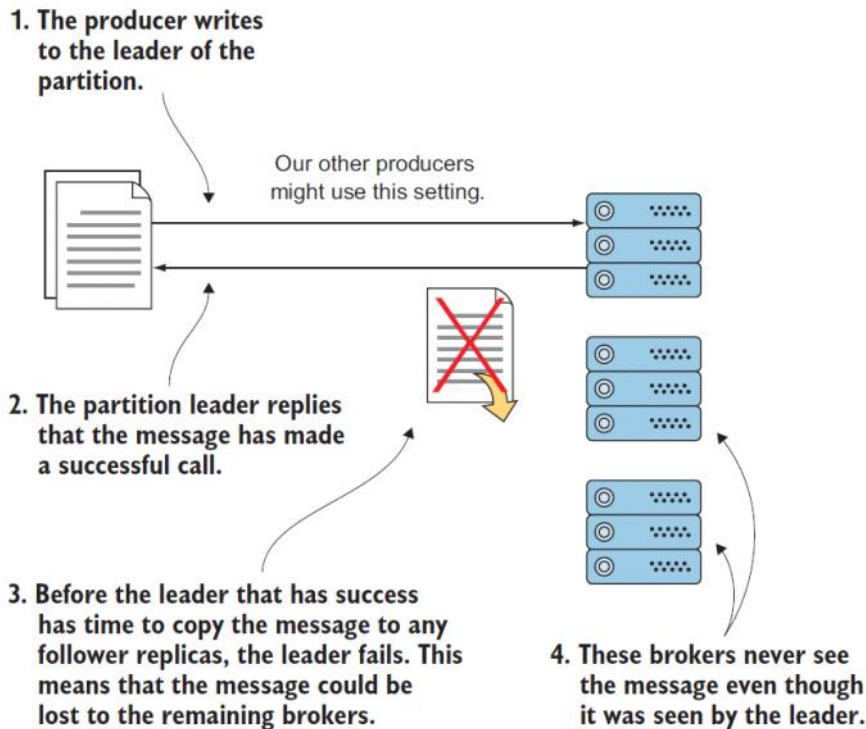


Figure 4.7 The property `acks equals 1`.

This is closely related to the ideas of at-most and at-least semantics that we covered in chapter 1 [10]. The setting is a part of that larger picture.

acks

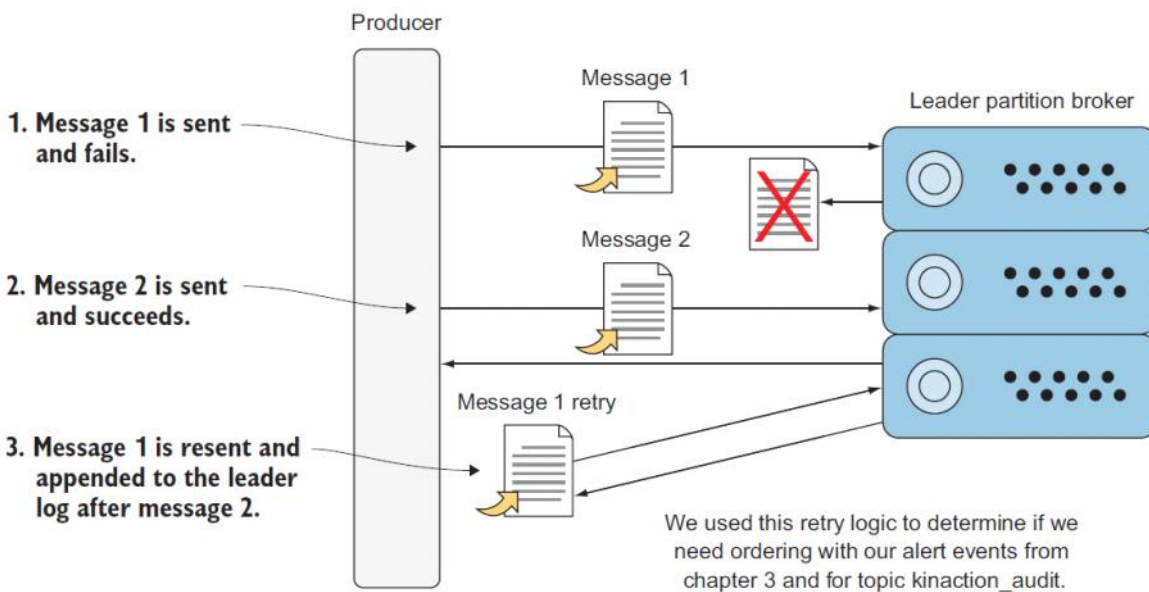


Figure 4.8 Retry impact on order

Chapter 5: Unlocking data

We use offsets as index positions in the log that the consumer sends to the broker. This lets the log know which messages it wants to consume and from where think back to our console consumer example, we used the flag `--from-beginning`. This sets the consumer's configuration parameter to `auto.offset.reset earliest` behind the scenes. With that configuration, you should see all the records for that topic for the partitions you are connected to, even if they were sent before you started the console consumer

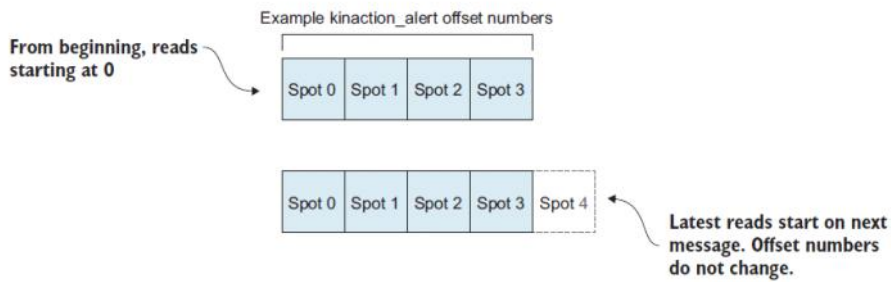


Figure 5.2 Kafka offsets [6]

Topic: 3 partitions, 2 replicas

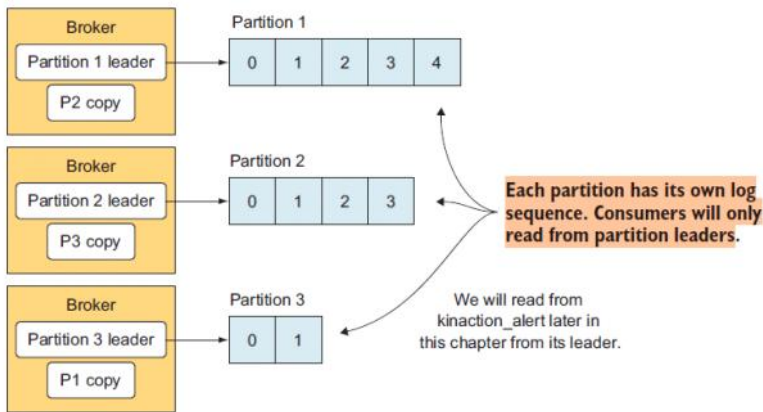


Figure 5.3 Partition leaders

Although consumers used ZooKeeper to store the offsets that they consume to a certain point, now the offsets are often stored inside a Kafka internal topic [10]. As a side note, consumer clients do not have to store their offsets in either of these locations, but this will likely be the case. If you want to manage your own offset storage you can! You can either store it in a local file, in cloud storage with a provider like AWS™, or a database. One of the advantages of moving away from ZooKeeper storage was to reduce the clients' dependency on ZooKeeper.

Consumers from different groups ignore each other, getting their own copy of the data.

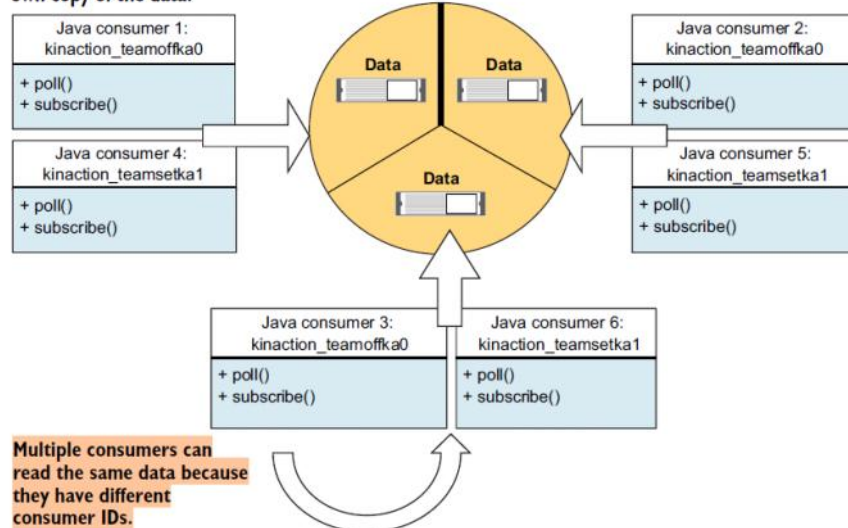


Figure 5.7 Consumers in separate groups [12]

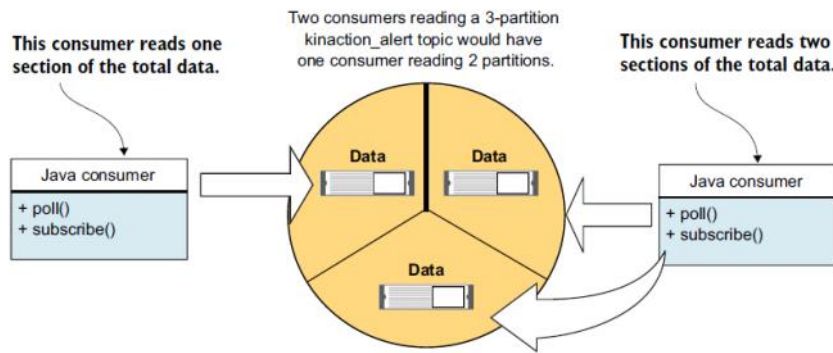


Figure 5.8 Kafka consumers in a group

Figure 5.8 highlights how one consumer can read two partitions leader replicas, where the second consumer can only read the data from a third partition leader [8]. A single partition replica is not to be divided or shared between more than one consumer with the same ID.

5.5 Reading from a compacted topic

how these topics work, but in short, we need to update records that have the same key value. If you do not need a history of messages, but rather just the last value, you might wonder how this concept works with an immutable log that only adds records to the end. The biggest “gotcha” for consumers that might cause an error is that when reading records from a compacted topic, consumers can still get multiple entries for a single key [22]! How is this possible? Because compaction runs on the log files that are on disk, compaction may not see every message that exists in memory during cleanup.

Clients need to handle this case, where there is more than one value per key. We should have the logic in place to handle duplicate keys and, if needed, ignore all but the last value. To pique your interest about compacted topics, note that Kafka uses its own compacted internal topic, called offsets, which relates directly to `__consumer__` your consumer offsets themselves [23]. Compaction makes sense here because for a specific combination of a consumer group, partition, and topic, only the latest value is needed as it will have the latest offset consumed.

Chapter 6

Brokers:

6.1 Introducing the broker

As we start to discover Kafka, those who are familiar with big data concepts or who have worked with Hadoop before might see familiar terminologies such as rack awareness (knowing which physical server rack a machine is hosted on) and partitions. Kafka has a **rack awareness** feature that makes replicas for a partition exist physically on separate racks [1].

6.2 Role of ZooKeeper

As ZooKeeper needs to have a minimum number in order to elect leaders and reach a decision, this cluster is indeed important for our brokers [3]. ZooKeeper itself holds information such as topics in our cluster [4]. ZooKeeper helps the brokers by coordinating assignments and notifications.

With all of this interaction with the brokers, it is important that we have ZooKeeper running before starting our brokers. The health of the ZooKeeper cluster impacts the health of our Kafka brokers. For instance, if our ZooKeeper instances are damaged, topic metadata and configuration could be lost

One example of this is version 3.1.x of Spring Cloud Stream, which allowed us to set the property [6]. The value defaulted to `zkNodes localhost` and should be left alone in most cases to avoid a ZooKeeper dependency. The property is marked as deprecated, but you never know if you will encounter `zkNodes` older code for maintenance, so you want to keep an eye out for it

Using the Kafka tool `sh`, which is located in the bin folder of `zookeeper-shell`. our Kafka installation, we can connect to a ZooKeeper host in our cluster and look at how the data is stored class [8]. In this file, you will find paths like `/controller/controller ZkData.scala_epoch,/config`, and `/brokers`, for example. If we look at the `/brokers/topics` path, we will see a list of the topics that we have created. At this point, we should, hopefully, at least have the topic in the list `kinaction_helloworld`

Listing 6.2 Listing our topics

```
bin/kafka-topics.sh --create \
  --bootstrap-server localhost:9094 \
  --topic kinaction_one_replica
```

Creates a topic with only one partition and one replica

```
bin/kafka-topics.sh --describe --bootstrap-server localhost:9094 \
  --topic kinaction_one_replica
```

```
Topic: one-replica PartitionCount: 1 ReplicationFactor: 1 Configs:
Topic: kinaction_one_replica Partition: 0
Leader: 2 Replicas: 2 Isr: 2
```

Describes the kinaction_one_replica topic with all the data located on the broker with ID 2

When we run the commands in listing 6.2 to create and describe the topic `_replica`, we'll see that there is only one value in the fields `Partition`, `kinaction_one Leader`, `Replicas`, and (in-sync replicas). Further, the broker uses the same ID `Isr` value. This means that the entire topic depends on that one broker being up and working.

If we terminate the broker with ID 2 in this example and then try to consume a message for that topic, we would get a message such as “1 partitions have leader brokers without a matching listener.” Because there are no replica copies for the topic's partition, there is no easy way to keep producing or consuming that topic without recovering that broker.

6.3.2 Server log

Many errors and unexpected behaviors can be traced back to configuration issues on startup. The server log file, `server.log`, is where we would look if there is a startup error or an exception that terminates the broker. It seems to be the most natural place to check first for any issues. Look (or use the command) for the heading `grep Kafka-values`. `Config`

6.3.3 Managing State

To figure out which broker is the current controller, we can use the `zookeeper-shell` script to look up the ID of the broker, as listing 6.3 shows. The path `/controller` exists in ZooKeeper, and in the listing, we run one command to look at the current value. Running that command for my cluster showed my broker with ID 0 as the controller

6.4 Partition replica leaders and their role

As a quick refresher, topics are made up of partitions, and partitions can have replicas for fault tolerance. Also, partitions are written on the disks of the Kafka brokers. One of the replicas of the partition will have the job of being the leader. The leader is in charge of handling writes from external producer clients for that partition. Because the leader is the only one with newly written data, it also has the job of being the source of data for the replica followers [15]. And because the ISR list is maintained by the leader, it knows which replicas are up to date and have seen all the current messages. Replicas act as consumers of the leader partition and will fetch the messages [15].

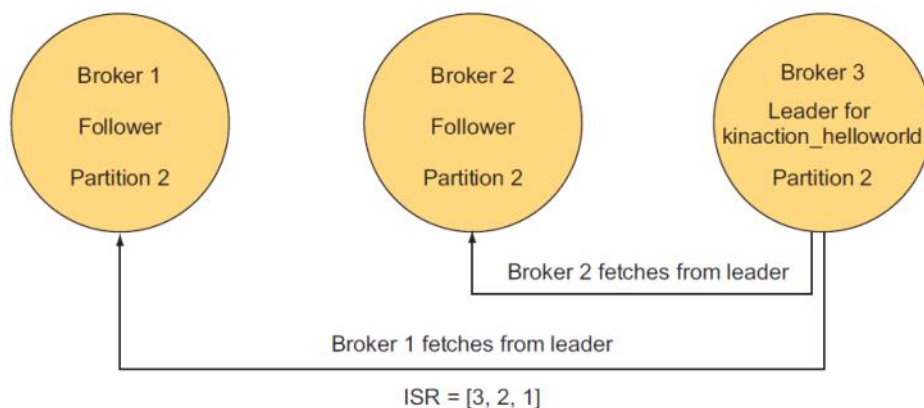


Figure 6.3 Leader

Because broker 3 is not available, a new leader is elected. Figure 6.4 shows the new leader broker 2. Once a follower, it was elected as a leader replica to keep Kafka serving and receiving data for that partition. The ISR list is now with the first position reflecting the `[2,1]` new leader replica hosted on broker 2.

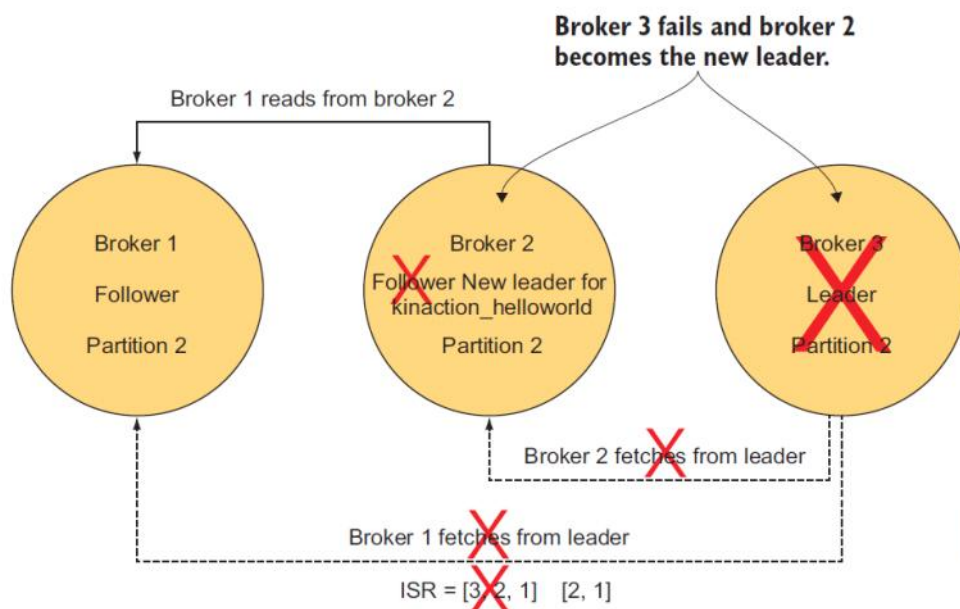


Figure 6.4 New leader elected

In chapter 5 we discussed a Kafka Improvement Proposal, KIP-392, which allows consumer clients to fetch from the closest replica [16]. Reading from a preferred follower rather than the leader replica is something that might make sense if our brokers span physical data centers. However, when discussing leaders and followers in this book, unless stated otherwise, we will focus on the default leader read and write behaviors.

It is also important to note that if a replica starts to get too far behind in copying messages from the leader, it can be removed from the ISR list. The leader notices if a follower is taking too long and drops it from its list of followers [17]. Then the leader continues to operate with a new ISR list. The result of this "slowness" to the ISR list is the same as in figure 6.4, in which a broker failed.

6.4.1 Losing Data

What if we have no ISRs and lose our lead replica due to a failure? When `unclean.leader.election.enable` is true, the controller selects a leader for a partition even if it is not up to date so that the system keeps running [15]. The problem with this is that data could be lost because none of the replicas have all the data at the time of the leader's failure.

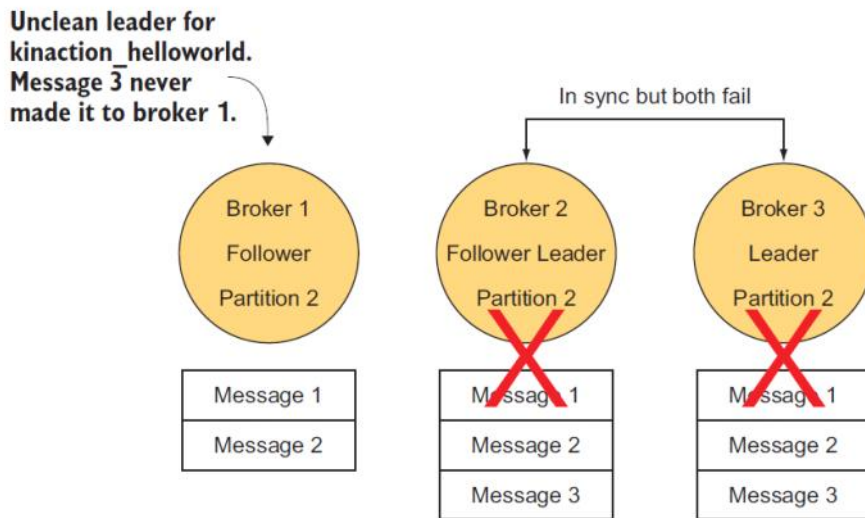


Figure 6.5 Unclean leader election

Broker 1 never sees message 3, so it cannot present that data to clients. At the cost of missing data, this option allows us to keep serving clients.

6.5 Peeking into Kafka

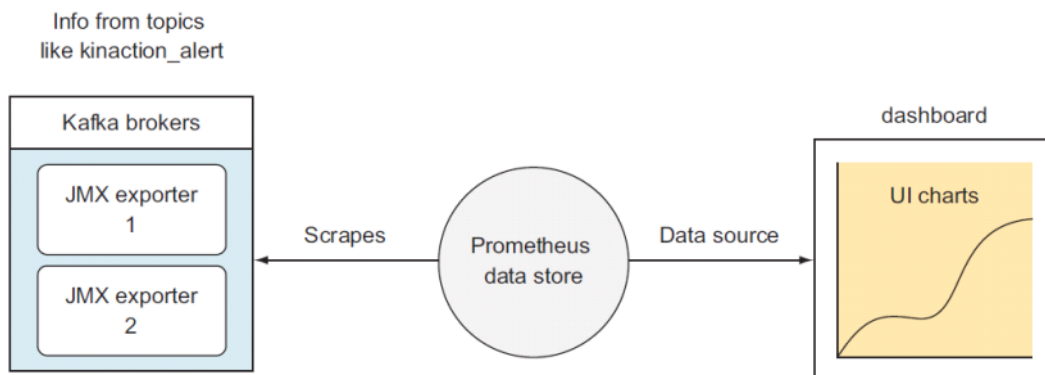


Figure 6.6 Graph flow

In figure 6.6, we use JMX to look inside the Kafka applications. The Kafka exporter takes the JMX notifications and exports them into the Prometheus format. Prometheus scrapes the exporter data and stores the metrics data. Various tools can then take the information from Prometheus and display that information in a visual dashboard

6.5.2 Adding a broker

Beginning with a small cluster is a great way to start, as we can always add brokers to grow our footprint. To add a Kafka broker to our cluster, we just start a new Kafka broker with a unique ID. This ID can either be created with the configuration `broker.id` or with `set` to [10]. That is pretty much it. But, `broker.id.generation.enable` true there is something to be aware of in this situation—the new broker will not be assigned to any partitions! Any topic partitions that we create before adding a new broker still persist on the brokers that existed at the time of their creation [20]. If we are okay with the new broker only handling new topics, then we don't need to do anything else

6.5.3 Upgrading your cluster

One technique that can be used to avoid downtime for our Kafka applications is the rolling restart [14]. This means just upgrading one broker at a time

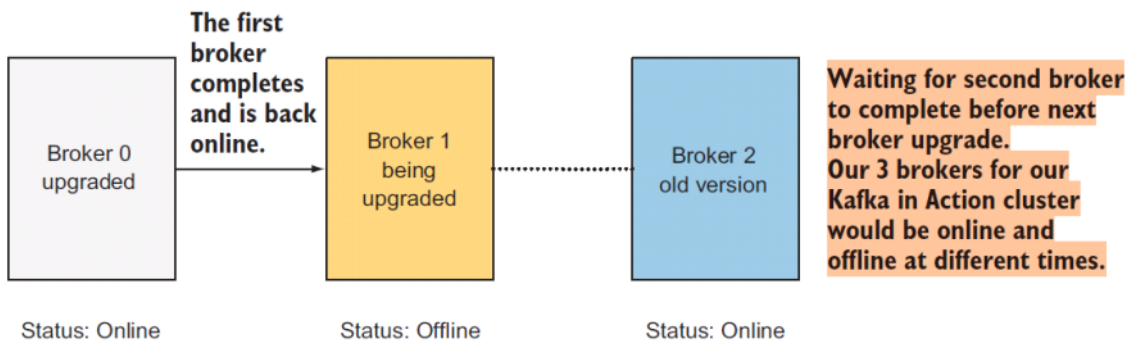


Figure 6.8 Rolling restart

6.5.5 Backups

Kafka does not have a backup strategy like one would use for a database; we don't take a snapshot or disk backup per se. Because Kafka logs exist on disk, why not just copy the entire partition directories? Although nothing is stopping us from doing that, one concern is making a copy of all of the data directories across all locations. Rather than performing manual copies and coordinating across brokers, one preferred option is for a cluster to be backed by a second cluster [23]. Between the two clusters, events are then replicated between topics. One of the earliest tools that you might have seen in production settings is MirrorMaker. A newer version of this tool (called Mirror-Maker 2.0) was released with Kafka version 2.4.0 [24]. In the bin subdirectory of the Kafka install directory, we will find a shell script named kafka-mirror-maker as well as a new MirrorMaker 2.0 script, connect-mirror-maker. There are also some other open source as well as enterprise offerings for mirroring data between clusters. Confluent Replicator (<http://mng.bz/Yw7K>) and Cluster Linking (<http://mng.bz/OQZo>) are also options to be aware of [25].

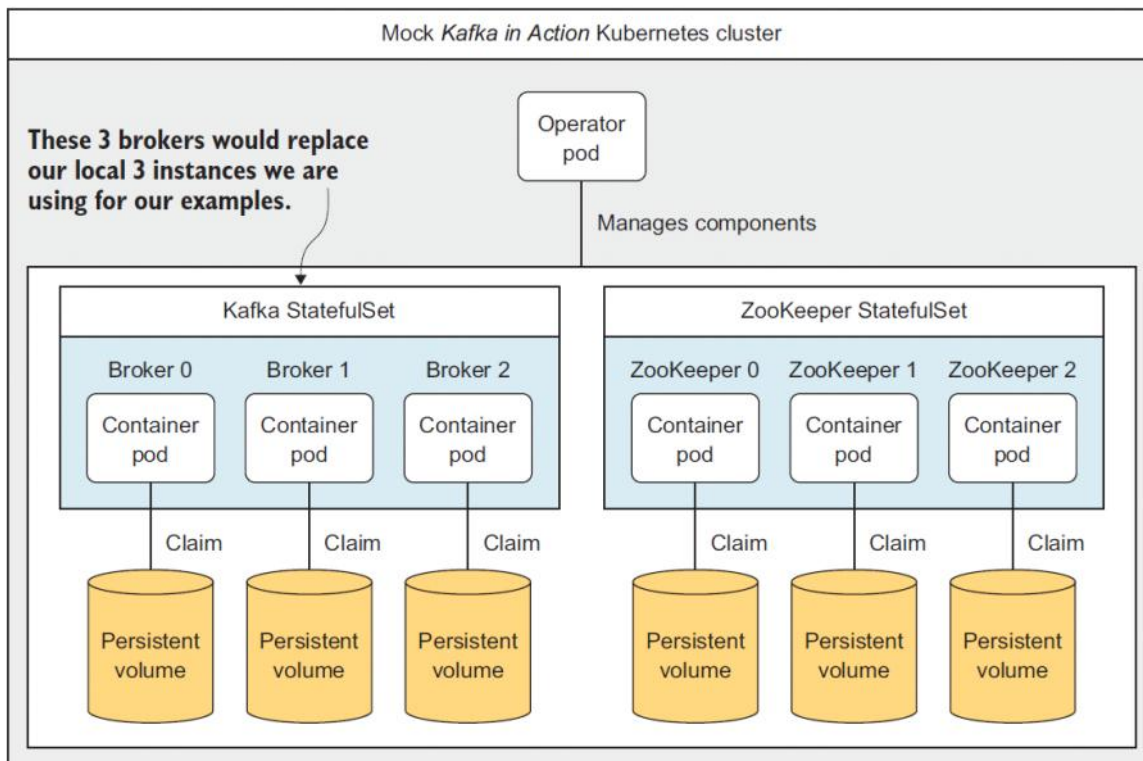


Figure 6.9 Kafka on Kubernetes

The Kubernetes operator is its own pod that lives inside of the Kubernetes cluster. As well, each broker is in its own pod as a part of a logical group called a StatefulSet. The purpose of the StatefulSet is to manage the Kafka pods and help guarantee ordering and an identity for each pod. If the pod that hosts a broker (the JVM process) with ID 0 fails, for example, a new pod is created with that identity (and not a random ID) and attaches to the same persistent storage volume as before. Because these volumes hold the messages of the Kafka partitions, the data is maintained. This statefulness helps overcome the sometimes short lives of containers. Each ZooKeeper node would also be in its own pod and part of its own StatefulSet.

Chapter 7: Topics and Partitions

7.1 Topics

We see topic design as a two-step process. The first looks at the events we have. Do they belong in one topic or more than one? The second considers each topic. What is the number of partitions we should use? The biggest takeaway is that partitions are a per-topic design question and not a cluster-wide limitation or mandate. Although we can set a default number of partitions for topic creation, in most cases, we should con

Sider how the topic will be used and what data it will hold

We have to take care of

- Message-keys
- Volume of messages
- Quantity of data

7.1.2 Replication factors

For practical purposes, we should plan on having the total number of replicas less than or equal to the number of brokers. In fact, attempting to create a topic with the number of replicas being greater than the total number of brokers results in an error: `InvalidReplicationFactorException`. Also, consumer clients cannot directly delete messages. This is what makes it possible to replay messages from a topic, which is a feature that can help us in many scenarios

7.2.1 Partition location

Its location can be found by looking for `log.dirs` in your `server.properties` file if you followed along from appendix A. Under that directory, we should be able to see subfolders with a topic name and a partition number. If we pick one of those folders and look inside, we will see a couple of different files with these extensions: `.index`, `.log`, and `.timeindex`. Figure 7.4 shows how a single partition (in this case, 1) in our test topic looks by issuing a directory listing (`ls`).

```
> ls /tmp/kafka/kafka-logs-0/kinaction_topicandpart-1
00000000000000000000.index  00000000000000000000.log  00000000000000000000.timeindex  leader-epoch-checkpoint
```

The files with the `.log` extension are where our data payload is stored. Other important information in the log file includes the offset of the message as well as the field. Why the need for any other files then? Because Kafka is built for CreateTime speed, it uses the `.index` and `.timeindex` files to store a mapping between the logical message offset and a physical position inside the index file [9].

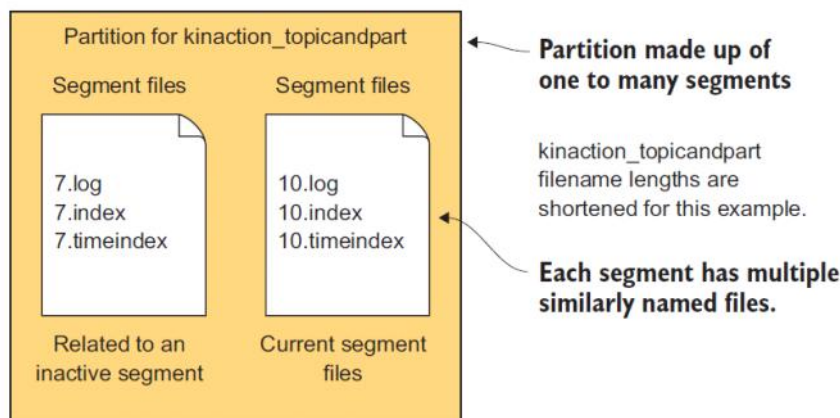


Figure 7.5 Segments make up a partition.

7.3 Testing with EmbeddedKafkaCluster

Kafka Streams provides an integration utility class called `EmbeddedKafkaCluster` that serves as a middle ground between mock objects and a full-blown cluster. This class provides an in-memory Kafka cluster [13]. Although built with Kafka Streams in mind, we can use it to test our Kafka clients. Listing 7.6 is set up like the tests found in the book *Kafka Streams in Action* by William P. Bejeck Jr., for example, his `KafkaStreamsYellingIntegrationTest` class [14]. That book and his following book, *Event Streaming with Kafka Streams and KSQLDB*, show more in-depth testing examples. We recommend checking those out, including his suggestion of using `Testcontainers` (<https://www.testcontainers.org/>). The following listing shows testing with `EmbeddedKafkaCluster` and `JUnit 4`.

7.3.1 Using Kafka TestContainers

NOTE One of the coauthors of this book, Viktor Gamov, maintains a repository (<https://github.com/gAmUssA/testcontainers-java-module-confluentplatform>) of integration testing Confluent Platform components (including Kafka, Schema Registry, KSQLDB).

7.4 Topic Compaction

With compaction, the goal is not to expire messages but rather to make sure that the latest value for a key exists and not to maintain any previous state. As just referenced, compaction depends on a key being part of the messages and that key not being null.

Let's say that we want to keep a current membership status for an online membership. A user can only be in one state at a time, either a Basic or a Gold membership. At first, a user enrolls in the Basic plan, but over time, upgrades to the Gold plan for more features. Although this is still an event that Kafka stores, in our case, we only want the most recent membership level for a specific customer (our key). Figure 7.7 shows an example using three customers

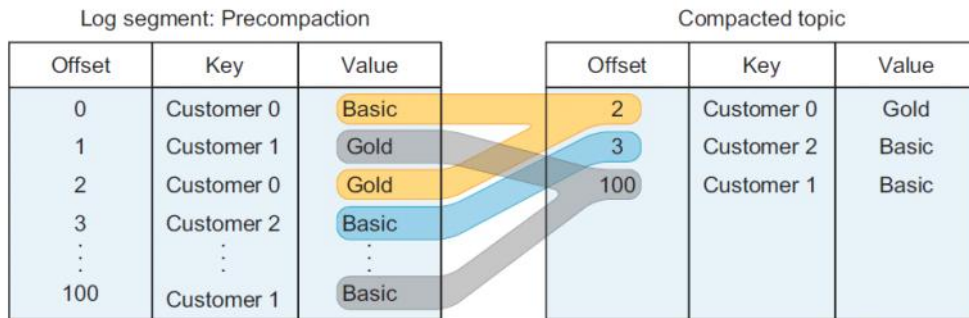


Figure 7.7 Compaction in general

Another real-world example of why one would want to use a compacted topic is Kafka's internal topic, `_offsets`. Kafka does not need a history of offsets __consumer that a consumer group consumes; it just needs the latest offset. By storing the offsets in a compacted topic, the log, in effect, gets an updated view of the current state of its world.

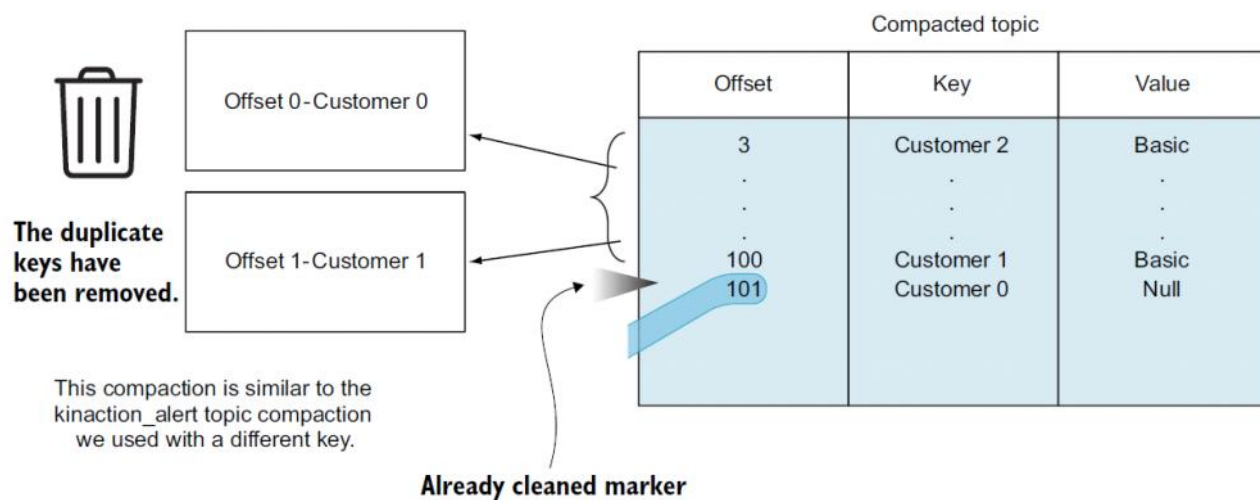


Figure 7.8 Compaction cleaning

Chapter 8: Kafka Storage

8.1 How long to store data

Currently, the default retention limit for data in Kafka topics is seven days, but we can easily configure this by time or data size [1]. But can Kafka hold data itself for a period of years? One real-world example is how the New York Times uses Kafka. The content in their cluster is in a single partition that was less than 100 GB at the time of writing [2]. If you recall from our discussion in chapter 7 about partitions, you know that all of this data exists on a single broker drive (as do any replica copies on their own drives) as partitions are not split between brokers.

How do we disable log retention limits and allow them to stay forever? By setting both `log.retention.ms` and `log.retention.bytes` to `-1`, we can effectively turn off data log retention. [4].

8.2.1 Keeping the original event

Having the original message makes it easier to go back and start over if you inadvertently messed up your transform logic. In stead of having to try to figure out how to fix your mistake on the altered data, you can always just go back to the original data and start again

8.2.2 Moving Away from a batch mindset

Does the general topic of ETL or data pipelines bring terms to mind such as batch, end of day, monthly, or even yearly? One of the shifts from the data transformation processes of the past is the idea that you can continuously stream your data into various systems without delay. With Kafka, for example, you can keep the pipeline running in nearreal time, and you can use its stream-processing platform to treat your data as an infinite series of events

8.3 Tools

8.3.1 Apache Flume

Look at the tools.

8.3.2 Red Hat Debezium

Look at the tools.

8.3.3 Secor

Look at the tools.

8.3.4 Example use case for data storage

Operational data is the events that are produced by our day-to-day operations. We can think of an event to order an item from a website as an example. A purchase event triggers our application into motion and does so in a low-latency way. The value of this data to our real-time applications might warrant keeping the data for a couple of days until the order is completed and mailed. After this timeframe, the event may become

more important for our analytical systems

Analytical data, while based on that same operational data, is usually used more to make business decisions. In traditional systems, this is where processes like a data warehouse, an online analytical processing system (OLAP), and Hadoop shine. That event data can be mined using different combinations of fields in our events in different scenarios to find insights into sales data, for instance. If we notice that sales of cleaning supplies always spike before a holiday, we might use that data to generate better sale options for our business in the future.

8.4 Bringing data back into Kafka

- Imagine a team working on trying to find patterns in data that they collected throughout years of handling events. In our example, there are terabytes of data. To serve operational real-time data collection, this data was moved from Kafka into HDFS after real-time consumers dealt with the messages. Does our application logic now have to pull from HDFS directly? Why not just pull it back into Kafka, and our application can process the data as it had before? Loading data into Kafka again is a valid way of reprocessing data that may have aged out of our system. Figure 8.4 shows another example of how we can move data back into Kafka.

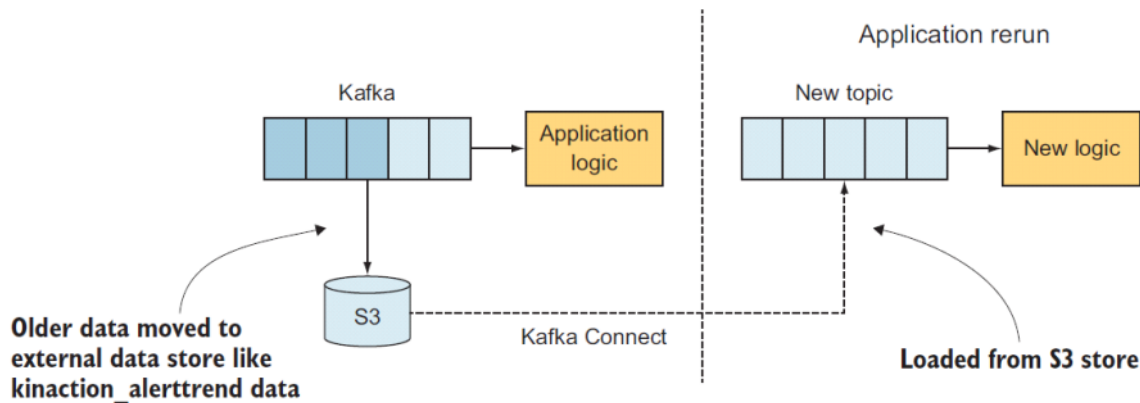


Figure 8.4 Moving data back into Kafka

8.4.1 Tiered storage

A newer option from the Confluent Platform version 6.0.0 on is called Tiered Storage. In this model, local storage is still the broker itself, and remote storage is introduced for data that is older (and stored in a remote location) and controlled by time configuration (confluent.tier.local.hotset.ms)

8.5.1 Lambda Architecture

The book Big Data, written by Nathan Marz with James Warren, discusses the lambda architecture more fully and goes into details about the batch, serving, and speed layers [9]. Figure 8.5 shows an example of how taking customer orders can be thought of in a batch and a real-time way. The customer totals from the previous days can be integrated with orders happening during the day into a combined data view to end users.

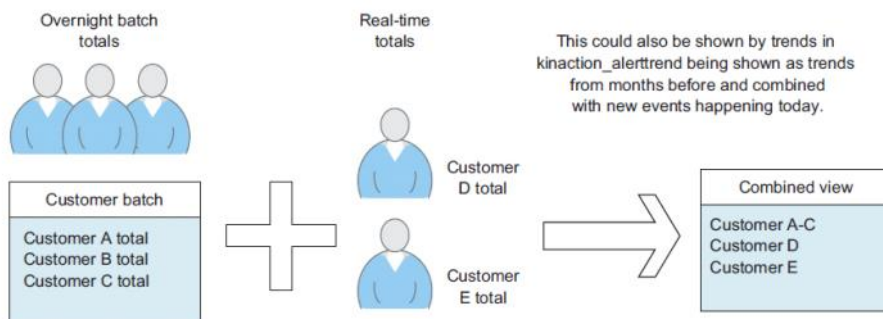


Figure 8.5 Lambda architecture

These layers are discussed in Big Data by Marz:

- Batch**—This layer is similar to the way batch processing with MapReduce occurs in a system like Hadoop. As new data is added to your data stores, the batch layer continues to precompute the view of the data that already lives in the system.
- Speed**—This layer is similar in concept to the batch layer except it produces views from recent data.
- Serving**—This layer updates the views it sends to consumers after each update to the batch views.

8.5.2 Kappa Architecture

Another way to do this is by running the current system in parallel to the new one and cutting over once the new version is ready to serve traffic. Part of this cutover is of course making sure that the data that is being served by the older version will be reflected correctly in the newer version. You only regenerate the user-facing data when you need to. There is no need to merge old and new data, which is an ongoing process for some lambda implementations. It does not have to be a continuous job, but rather invoked when you need an application logic change. Also, there's no need to change your interface to your data. Kafka can be used by both your new and old application code at the same time. Figure 8.6 shows how customer events are used to create a view without using a batch layer. Figure 8.6 shows customer events from the past and present being used directly to create a view. Imagine the events being sourced from Kafka and then using Kafka Streams or sqldb to read all the events in near-real time and creating a view for end users.

If a change is ever needed to how customer events are processed, a second application can be created with different logic (like a new sqldb query), using the same data source (Kafka) as before. There is no need to have a batch layer (and manage it) as there is only streaming logic used for making your end user views.

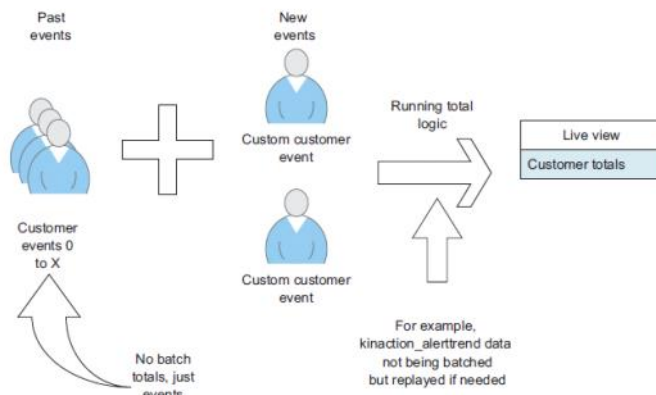


Figure 8.6 Kappa architecture

8.6 Multiple cluster setups

8.6.1 Scaling by adding clusters

Usually, the first things to scale would be the resources inside your existing cluster. The number of brokers is the first option that makes a straightforward path to growth. X Netflix®'s multicluster strategy is a captivating take on how to scale Kafka clusters [11]. Instead of using only the broker number as the way to scale the cluster, they found they could scale by adding clusters themselves!

Chapter 9:

Management: Tools and logging

9.1.1 Administration in code with AdminClient

AdminClient is a great tool for building a user-facing application for those who wouldn't normally need or want to use the Kafka shell scripts. It also provides a way to control and monitor what is being done on the cluster.

9.1.2 kcat

(<https://github.com/edenhill/kcat>) is a handy tool to have on your workstation, Kcat especially when connecting remotely to your clusters. At this time, it focuses on being a producer and consumer client that can also give you metadata about your cluster. If you ever want to quickly work with a topic and don't have the entire Kafka toolset downloaded to your current machine, this executable helps you avoid the need to have those shell or bat scripts

9.3.1 Application logs

Note that modifying kafkaAppender changes only how the server.log file is treated. If we want to apply different file sizes and backup file numbers for various Kafka-related files, we can use the appender to log a filename table to determine which appenders to update. In table 9.1, the appender name in the left column is the logging key, which affects how the log files on the right are stored on the brokers [8].

Table 9.1 Appender to log pattern

Appender name	Log filename
kafkaAppender	server.log
stateChangeAppender	state-change.log
requestAppender	kafka-request.log
cleanerAppender	log-cleaner.log
controllerAppender	controller.log
authorizerAppender	kafka-authorizer.log

9.3.2 ZooKeeper logs

The default configuration for ZooKeeper does not remove log files, but our Kafka install may have added that feature for us. If you followed the setup of our local ZooKeeper node from appendix A, these values can be set in the file config/zookeeper.properties. Either way, it is a good idea to make sure that the retention of the ZooKeeper application logs are controlled by the following configuration values and are what we need for troubleshooting:

9.4 Firewalls

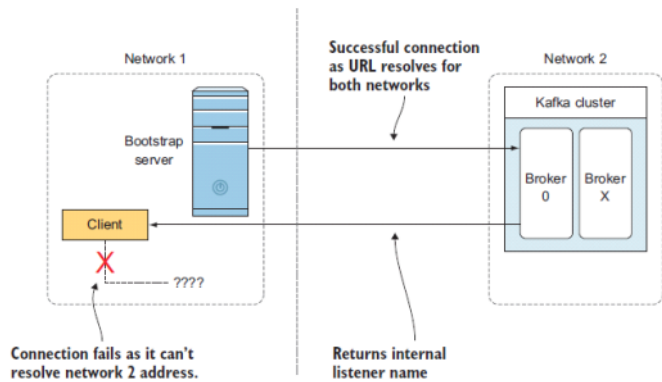
For example, the default for a plain text port is 9092. An SSL port at 9093 can also be set up on that same host. Both of these ports might need to be open, depending on how clients connect to our brokers. In addition, ZooKeeper includes port 2181 for client connections. Port 2888 is used by follower ZooKeeper nodes to connect to the leader ZooKeeper node, and port 3888 is also used between ZooKeeper nodes to communicate [11].

9.4.1 Advertised Listeners

Let's imagine we are connecting to a broker and can get a connection when the client starts, but not when it attempts to consume messages. How is this behavior that appears inconsistent possible? Remember that when a client starts, it connects to any broker to get metadata about which broker to connect to. The initial connection from the client uses the information that is located in the Kafka configuration. Listeners What it gives back to the client to connect to next is the data in Kafka's advertised [12]. This makes it likely that the client will connect to a different host to .listeners do its work.

Figure 9.3 shows how the client uses one hostname for the first connection attempt, then uses a different hostname on its second connection. This second host name was given to the client from its initial call as the new location to connect to.

Scenario 1: no advertised listeners. Producer client starts and requests metadata from bootstrap server.



Scenario 2: advertised listeners with URL resolved by both networks. Producer client requests metadata.

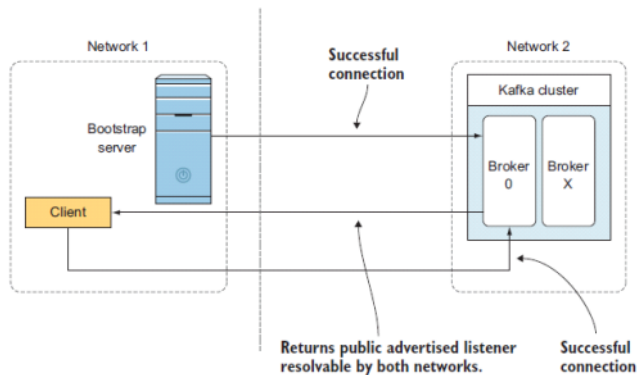


Figure 9.3 Kafka's advertised listeners compared to listeners

An important setting to look at is `inter.broker.listener.name`, which determines how the brokers connect across the cluster to each other [12]. If the brokers cannot reach each other, replicas fail and the cluster will not be in a good state, to say the least! For an excellent explanation of advertised listeners, check out the article by Robin Moffatt, "Kafka Listeners – Explained," if you want to dig into more details [12]. Figure 9.3 was inspired by Robin Moffatt's diagrams on that site as well

9.5.1 JMX Console

VisualVM (<https://visualvm.github.io/>) is one example. Looking at the available JMX metrics can help us discover points of interest in which we might choose to add alerts. When installing VisualVM, be sure to go through the additional step of installing the MBeans Browser [13]. Make sure that you correctly scope it to be separate for `export JMX_PORT=49999` each broker as well as each ZooKeeper node. is also another option to look at for connecting remotely to Kafka KAFKA_JMX_OPTS brokers. Make sure to note the correct port and hostname. Listing 9.10 shows an example that sets with various arguments [13]. It uses port 49999 KAFKA_JMX_OPTS and the localhost as the hostname. In the listing, the other parameters allow us to connect without SSL and to not have to authenticate